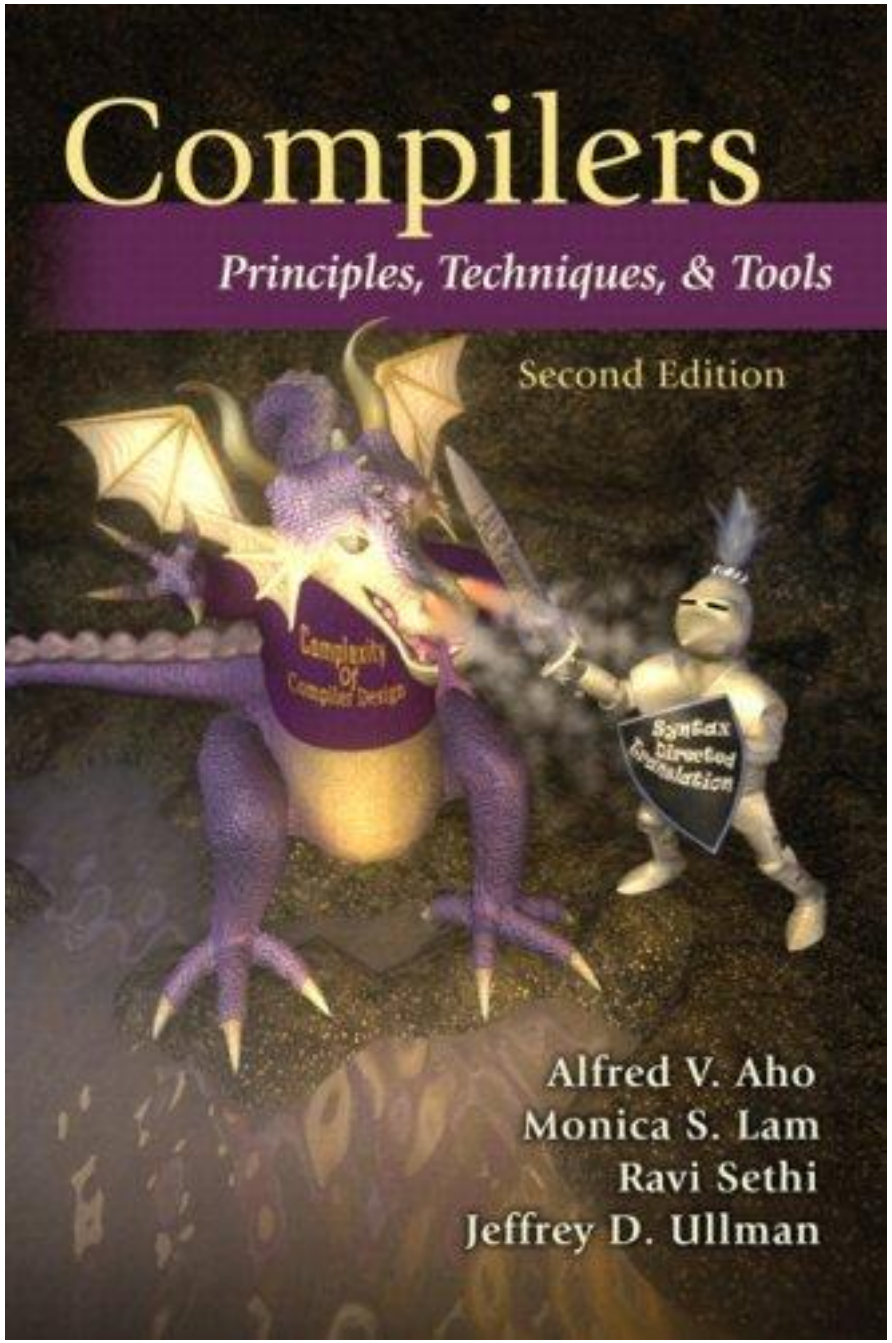
四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn



Chapter 4

Syntax Analysis

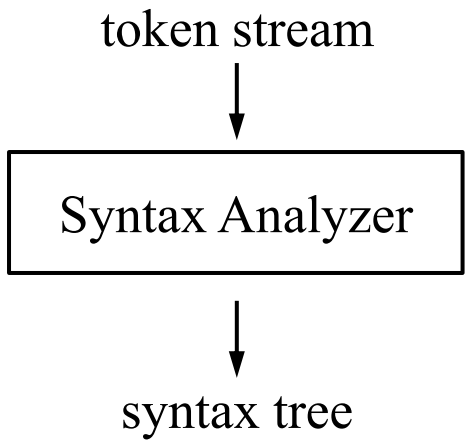
Chapter 4

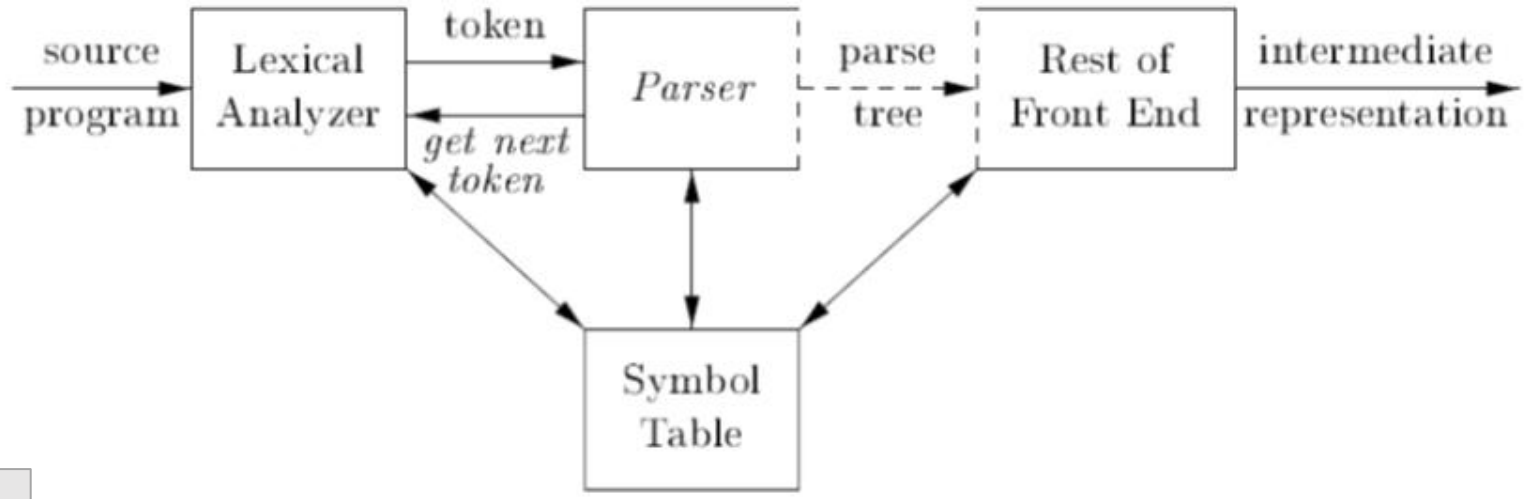
- 上下文无关文法
- 自顶向下分析：递归下降分析，LL(1)分析
- 自底向上分析：LR(0)分析，SLR(1)分析，LR(1)分析，LALR(1)分析

CONTENTS

4.1 Introduction

4.1.1 The Role of the Parser






```
TreeNode* parse(void)
{
    TreeNode * t;
    token = getToken();
    ……
    return t;
}
```

- Types of Parsers

Universal	Some universal methods can parse any grammar, but too inefficient to use in production compilers.
Top-down	Recursive-descent parsing, LL(1), etc.
Bottom-up	LR(0), SLR(1), LR(1), LALR(1), etc.

4.1.2 Representative Grammars

- 4.2 Context-Free Grammars

4.1.3 Syntax Error Handling

- Lexical errors
- Syntactic errors
- Semantic errors
- Logical errors

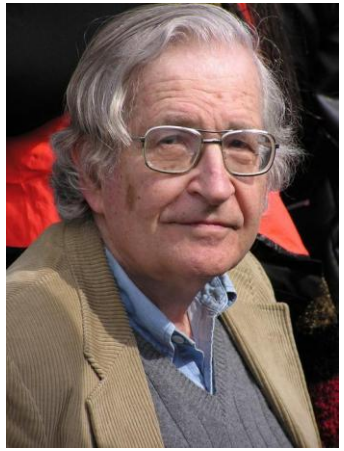
```
int x;
double n = .123E10.0, x;

x = n++;
x = 1+;
if (x = 0) x++;
```

4.2 Context-Free Grammars

Chomsky Hierarchy

- Four kinds of grammars
  - type 0：unrestricted grammar
  - type 1：context sensitive grammar
  - type 2：context free grammar
  - type 3：regular grammar equivalent to regular expression

```
position = initial + rate * 60
```



诺姆·乔姆斯基（Noam Chomsky）简介

基本信息

- **全名**：Avram Noam Chomsky
- **出生日期**：1928年12月7日
- **出生地**：美国宾夕法尼亚州费城
- **职业**：语言学家、哲学家、认知科学家、政治评论家

“→”表示“由…组成”或“定义为”

<句子> → <主语><谓语><间接宾语><直接宾语>
<主语> → <代词>
<谓语> → <动词>
<间接宾语> → <代词>
<直接宾语> → <冠词> <名词>
<代词> → He
<动词> → gave
<代词> → me
<冠词> → a
<名词> → book

“He gave me a book”是不是一个合法的英语句子?

<句子> ⇒ <主语><谓语><间接宾语><直接宾语>
⇒ <代词><谓语><间接宾语><直接宾语>
⇒ He <谓语><间接宾语><直接宾语>
⇒ He <动词><间接宾语><直接宾语>
⇒ He gave <间接宾语><直接宾语>
⇒ He gave <代词><直接宾语>
⇒ He gave me <直接宾语>
⇒ He gave me <冠词><名词>
⇒ He gave me a <名词>
⇒ He gave me a book

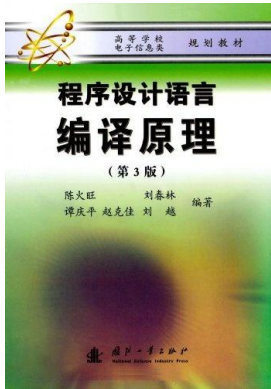
- 推导（derivation）

“He gave me a book”是一个合法的英语句子。

- 上述定义英文句子的规则就是一个上下文无关文法。其中，
  - He, me, book, gave, a 等，称为**终结符号**（terminal）；
  - <句子>、<主语>等，称为**非终结符号**（nonterminal）；
  - <句子>称为**开始符号**（start symbol）；
  - 规则如<主语> → <代词>等称为**产生式**（production）。

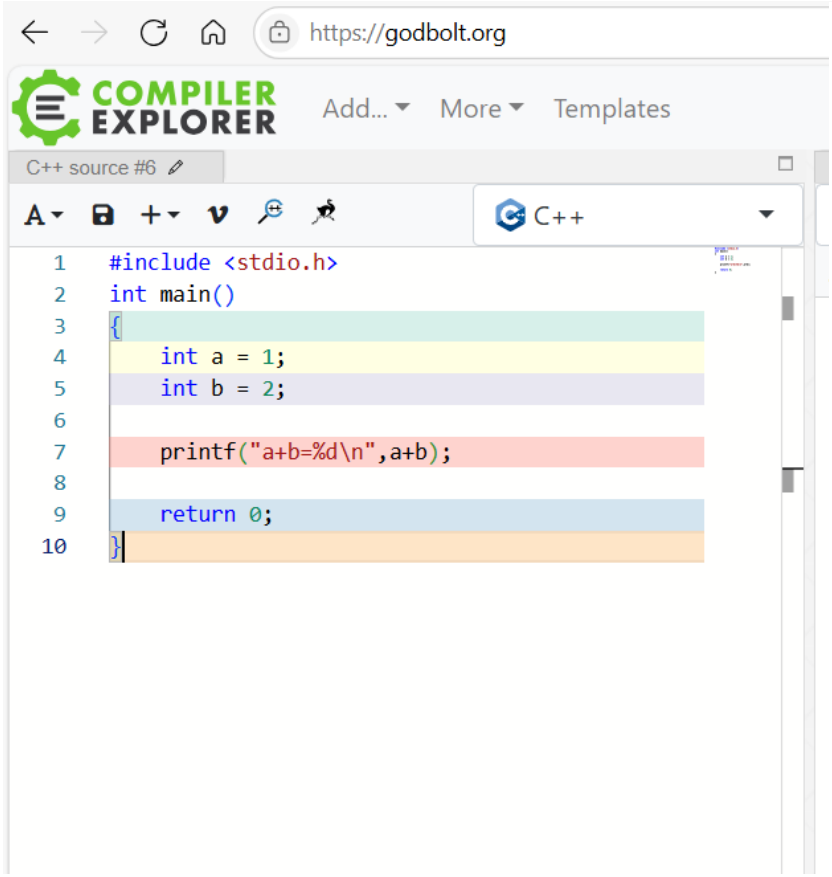
1. V_T
2. V_N
3. S
4. P

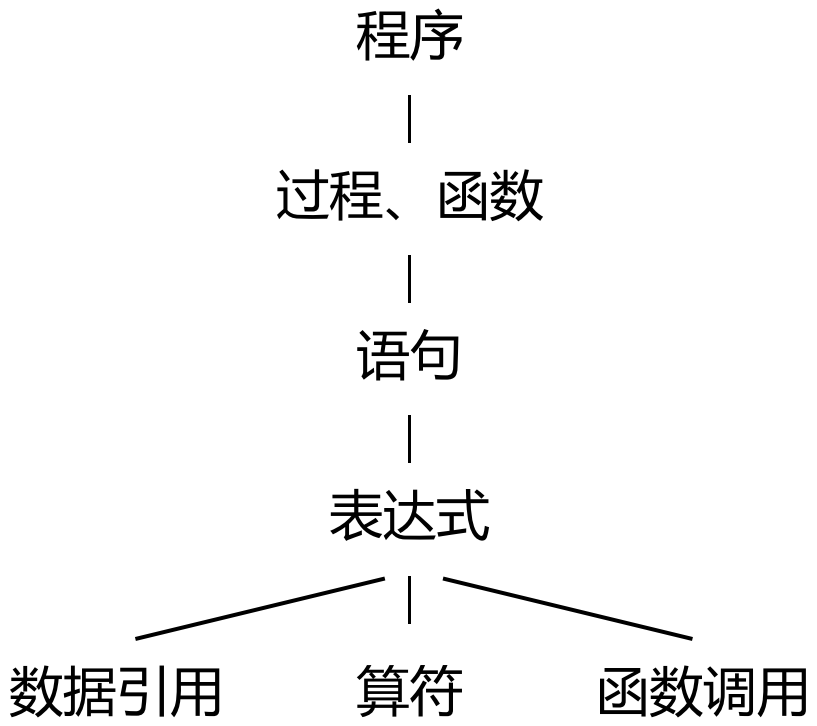
$< V_T, V_N, S, P >$



Structure of a Programming Language

- A context-free grammar is a specification for the syntactic structure of a programming language.





Definition for Arithmetic Expressions

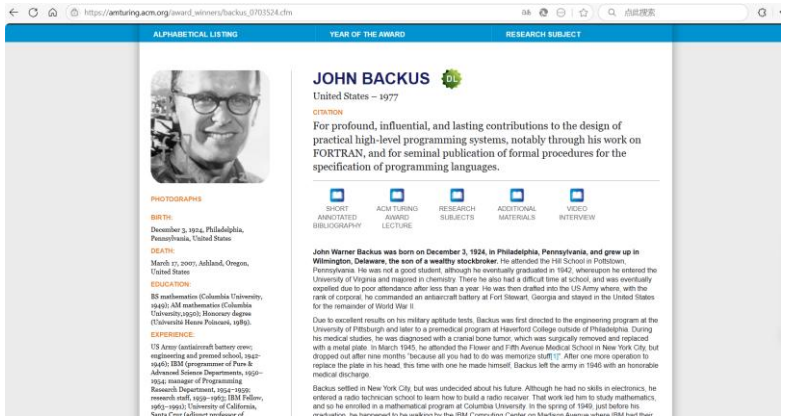
<table>
<tr><td rowspan="3">Rules</td><td>1. Variable i is an arithmetic expression;</td></tr>
<tr><td>Operations: +, *, and ( ) to change priority;</td></tr>
<tr><td>2. If E_1 and E_2 are arithmetic expressions, then
E_1+E_2, E_1*E_2 , and (E_1) are arithmetic expressions.</td></tr>
<tr><td rowspan="2">Grammar
$< V_T, V_N, S, P >$</td><td>$G = < \{ \mathbf{i}, +, *, (,) \} , \{E\}, E, P >$</td></tr>
<tr><td>$E \rightarrow \mathbf{i}$
$E \rightarrow E+E$
$E \rightarrow E*E$
$E \rightarrow (E)$
} recursion (direct, indirect)
Production: head → body or left side → right side</td></tr>
</table>

4.2.1 The Formal Definition of a Context-Free Grammar

- $G = < V_T, V_N, S, P >$

1. terminals V_T
2. nonterminals V_N, and $V_T \cap V_N = \varnothing$
3. start symbol $S \in V_N$
4. productions $P \rightarrow \alpha, \quad P \in V_N, \ \alpha \in (V_T \cup V_N)^*$

- Backus-Naur Form (BNF)
  - 巴科斯范式是描述语法规则的一种形式化表示方法，由John Backus 和 Peter Naur 提出。
  - 采用这种方式定义语法规则，可以用简洁的公式把各种语法规则严格而清晰地描述出来。



4.2.2 Notational Conventions

- V_T : Lowercase letters, digits, operators, punctuations, boldface strings such as **id** or **if**. tokens
- V_N : Uppercase letters such as $E \rightarrow E + T$; Lowercase, italic names such as $exp \rightarrow exp + term$
- Start symbol
  - S is usually the start symbol.
  - G[N]: $N \rightarrow D \mid ND$
    $D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$
  - Unless stated otherwise, the head of the first production is the start symbol.
    $D \rightarrow T\ L \qquad T \rightarrow \mathbf{int} \mid \mathbf{float} \qquad L \rightarrow L, \mathbf{id} \mid \mathbf{id}$
- α, β, γ represent strings of grammar symbols. $\alpha, \beta, \gamma \in (V_T \cup V_N)^*$
- Call each α_i the alternatives for P.

$$P \rightarrow \alpha_1 \mid P \rightarrow \alpha_2 \mid \cdots \mid P \rightarrow \alpha_n \quad \Rightarrow \quad P \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$$

$$E \rightarrow i \mid E \rightarrow E{+}E \mid E \rightarrow E{*}E \mid E \rightarrow (E) \quad \Rightarrow \quad E \rightarrow i \mid E{+}E \mid E{*}E \mid (E)$$

4.2.3 Derivations

- **推导**：从开始符号出发，连续使用产生式，对非终结符进行替换。
  （每一步只能用一个产生式替换一个非终结符）
- The symbol ⇒ means, “derives in one step.”

<句子> ⇒ <主语><谓语><间接宾语><直接宾语>
⇒ <代词><谓语><间接宾语><直接宾语>
⇒ He <谓语><间接宾语><直接宾语>
⇒ He <动词><间接宾语><直接宾语>
⇒ He gave <间接宾语><直接宾语>
⇒ He gave <代词><直接宾语>
⇒ He gave me <直接宾语>
⇒ He gave me <冠词><名词>
⇒ He gave me a <名词>
⇒ He gave me a book

- Consider the grammar

$$E \rightarrow \mathbf{i} \mid E+E \mid E*E \mid (E)$$

- Derivation for $(\mathbf{i}+\mathbf{i})$

- $(\mathbf{i}+\mathbf{i})$ is a legal sentence of the grammar.

Example

- Consider the following grammars.

$G_1[S]$: $S \rightarrow bA$

$A \rightarrow aA \mid a$

- Derivation for baaa
- Other derivations

$S \Rightarrow bA \Rightarrow ba$

$S \Rightarrow bA \Rightarrow baA \Rightarrow baa$

…

$S \Rightarrow bA \Rightarrow baA \Rightarrow \ldots \Rightarrow baa\ldots a$

- The language generated by $G_1[S]$

$L(G_1) = \{ ba^n \mid n \geqslant 1 \}$

- Consider the following grammars.

$G_2[S]$: $S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

- Derivation for abb
- Other derivations

$S \Rightarrow AB \Rightarrow aB \Rightarrow ab$

$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabbB \Rightarrow aabbb$

…

$S \Rightarrow AB \Rightarrow \ldots \Rightarrow aa\ldots abb\ldots b$

- The language generated by $G_2[S]$

$L(G_2) = \{ a^m b^n \mid m, n \geqslant 1 \}$

Definition

- 直接推导
  - 如果 $A \to \gamma$ 是一个产生式，且α，$\beta \in (V_T \cup V_N)^*$，则将产生式 $A \to \gamma$ 用于符号串$\alpha A \beta$，得到符号串$\alpha\gamma\beta$，记为$\alpha A \beta \Rightarrow \alpha\gamma\beta$，称$\alpha A \beta$直接推导出$\alpha\gamma\beta$。
- 推导（derivation）
  - 如果$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \ldots \Rightarrow \alpha_n$，则称该序列是从$\alpha_1$到$\alpha_n$的一个推导。
  - 如果存在一个从α_1到α_n的推导，则称α_1可以推导出α_n。
  - 用 $\alpha_1 \Rightarrow^+ \alpha_n$ 表示：从α_1出发，经过一步或若干步，可以推出α_n 。
  - 用 $\alpha_1 \Rightarrow^* \alpha_n$ 表示：从α_1出发，经过零步或若干步，可以推出α_n 。
- sentential form & sentence
  - If $S \Rightarrow^* \alpha$, where S is the start symbol of a grammar G, and $\alpha \in (V_T \cup V_N)^*$, we say that α is a sentential form of G.
  - If $\alpha \in V_T^*$, we say that α is a sentence of G.
- The language generated by a grammar G is its set of sentences. $L(G) = \{ \alpha \mid S \Rightarrow^* \alpha \ \& \ \alpha \in V_T^* \}$.
- If two grammars generate the same language, the grammars are equivalent.

Example

- Describe the languages generated by the following grammars.

1. $N \rightarrow D \mid ND$
   $D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

2. $N \rightarrow D \mid DN$
   $D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

3. $E \rightarrow (E)$

   $L(E) = \{\ \}$ or $L(E) = \Phi$

1.可以以0开头的数字串
2.可以以0开头的数字串
3.因为没有终结符，所以说推导过程将无限进行无法终止，因此生成的是空语言

- Design a grammar G for the following language.

$L(G_3) = \{ a^n b^n \mid n \geqslant 1 \}$ L(G3): S -> aSb | ab

$L(G_4) = \{ (^n a)^n \mid n \geqslant 0 \}$ L(G4): S -> (S) | a

- Design a grammar equivalent to the regular expression a*. L(G5): S -> aS | ε
- ε-production: $A \rightarrow \varepsilon$

Left Recursive and Right Recursive

left recursive $A \Rightarrow^+ A\alpha$	right recursive $A \Rightarrow^+ \alpha A$
$A \to A\alpha$ direct left recursive	$A \to \alpha A$ direct right recursive
$A \to A\alpha \mid \beta$	$A \to \alpha A \mid \beta$

indirect left recursive

$S \to Qc \mid c$

$Q \to Rb \mid b$

$R \to Sa \mid a$

$S \Rightarrow Qc \Rightarrow Rbc \Rightarrow Sabc$

$E \to i \mid E+E \mid E*E \mid (E)$

Leftmost Derivation and Rightmost Derivation

- The derivation from a sentential form to another is usually not unique.
- $E+E+E \Rightarrow^{+} i+i+i$

$E+E+E \Rightarrow i+E+E \Rightarrow i+i+E \Rightarrow i+i+i$	$E+E+E \Rightarrow E+E+i \Rightarrow E+i+i \Rightarrow i+i+i$
• leftmost derivation $\alpha \underset{lm}{\Rightarrow} \beta,\ \alpha \underset{lm}{\Rightarrow}^{*} \beta$	• rightmost derivation $\alpha \underset{rm}{\Rightarrow} \beta,\ \alpha \underset{rm}{\Rightarrow}^{*} \beta$
• left-sentential form $S \underset{lm}{\Rightarrow}^{*} \alpha$	• right-sentential form $S \underset{rm}{\Rightarrow}^{*} \alpha$

- 非最左/最右推导：$E+E+E \Rightarrow E+i+E$ 或 $E+E+E \Rightarrow i+E+E \Rightarrow i+E+i$
- 为了对句子结构进行确定性的分析，通常只考虑最左推导或最右推导。
- Consider the grammar G(N):

  $N \rightarrow D \mid ND$

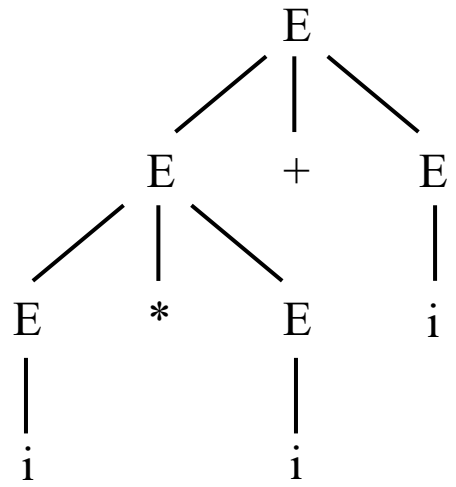
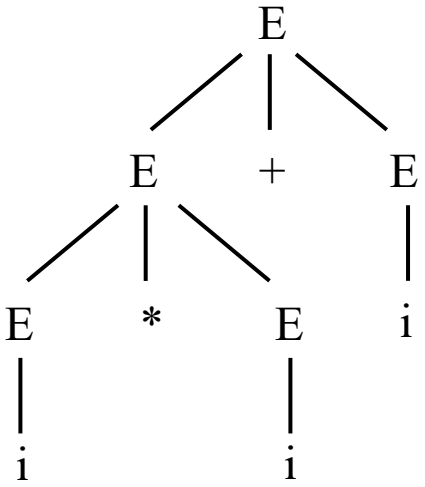
  $D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

  - Give a leftmost derivation for the sentence 0123.
  - Give a rightmost derivation for the sentence 0123.

最左推导：
S => ND
=> NND
=> NNND
=> DNND
=> 0NND
=> 0DND
=> 01ND
=> 01DD
=> 012D
=> 0123

4.2.4 Parse Trees

- 语法分析树是推导的图形化表示形式，通常简称语法树或分析树。
- $E \rightarrow i \mid E+E \mid E*E \mid (E)$ Derivation for i*i+i

leftmost derivation		rightmost derivation	
$E \Rightarrow E+E$ $\Rightarrow E*E+E$ $\Rightarrow i*E+E$ $\Rightarrow i*i+E$ $\Rightarrow i*i+i$			$E \Rightarrow E+E$ $\Rightarrow E+i$ $\Rightarrow E*E+i$ $\Rightarrow E*i+i$ $\Rightarrow i*i+i$

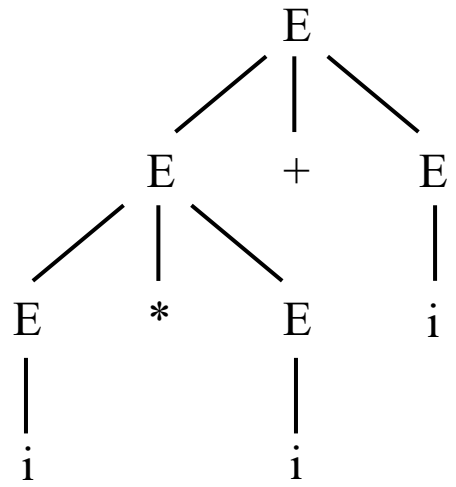
- Relationship between derivations and parse tree: many-to-one
  - Since a parse tree ignores variations in the order in which symbols in sentential forms are replaced.
- Relationship between parse trees and leftmost/rightmost derivation: one-to-one
  - Both leftmost and rightmost derivations pick a particular order for replacing symbols in sentential forms.

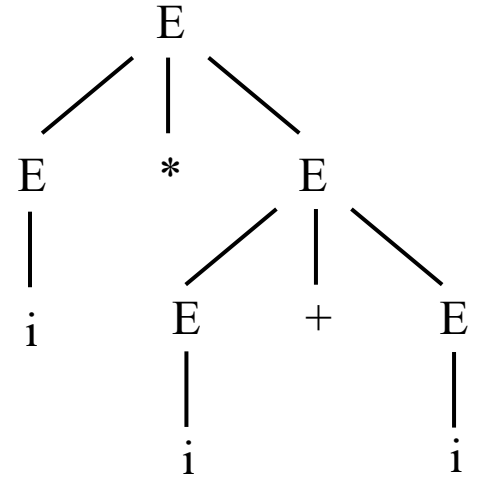
Relationship between sentences and parse trees

- $E \rightarrow i \mid E+E \mid E*E \mid (E)$

a sentence ? parse tree

leftmost derivation	leftmost derivation
$E \Rightarrow E+E$ $\Rightarrow E*E+E$ $\Rightarrow i*E+E$ $\Rightarrow i*i+E$ $\Rightarrow i*i+i$	$E \Rightarrow E*E$ $\Rightarrow i*E$ $\Rightarrow i*E+E$ $\Rightarrow i*i+E$ $\Rightarrow i*i+i$





4.2.5 Ambiguity

- A grammar that produces more than one parse tree for some sentence is said to be ambiguous.
- 在语言理论中已经证明，二义性问题是不可判定问题。

  即不存在一个算法，它能在有限步骤内，确切地判定一个文法是否是二义性的。
- Solution: Find a set of sufficient conditions for an unambiguous grammar and rewrite the ambiguous grammar to be unambiguous.
- Ambiguity of Expression:

$$E \rightarrow i \mid E+E \mid E*E \mid (E)$$

i*i+i　　i+i+i

Precedence (2.2.6) & Associativity (2.2.5)

- Solution for precedence: Showing the operators in order of increasing precedence.
- Solution for associativity: Keep only one recursion.

$E \rightarrow i \mid E+E \mid E*E \mid (E)$ → $E \rightarrow E+E \mid T$, $T \rightarrow T*T \mid F$, $F \rightarrow (E) \mid i$ →

左递归实现左结合	右递归实现右结合
$E \rightarrow E+T \mid T$	$E \rightarrow T+E \mid T$
$T \rightarrow T*F \mid F$	$T \rightarrow F*T \mid F$
$F \rightarrow (E) \mid i$	$F \rightarrow (E) \mid i$

- **Example 4.5 & 4.6** Grammars for simple arithmetic expressions

expression → *expression* + *term* | *expression* - *term* | *term*
term → *term* * *factor* | *term* / *factor* | *factor*
factor → **(***expression***)** | **id**

$E \rightarrow E + T \mid E - T \mid T$
$T \rightarrow T * F \mid T / F \mid F$
$F \rightarrow (E) \mid id$

Ambiguity of If-Statement

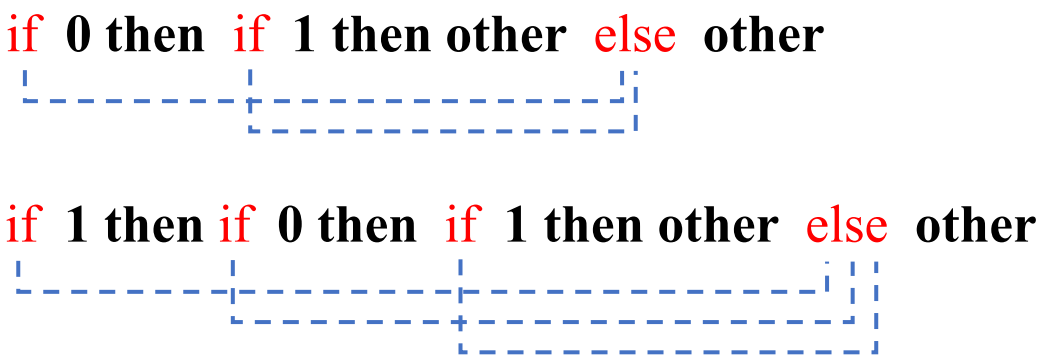
- "dangling-else" grammar

$S \rightarrow$ **if** E **then** S

| **if** E **then** S **else** S

| **other**

$E \rightarrow$ **0** | **1**



- Solution: rewrite the grammar (4.3.2)
  - 基本思想： if-else之间只能是已匹配的if-else语句或其他语句。

$S \rightarrow S_1 \mid S_2$ （匹配句 | 非匹配句）

$S_1 \rightarrow$ if E **then** S_1 else S_1 | **other**

$S_2 \rightarrow$ **if** E **then** S | if E **then** S_1 else S_2

Another Solutions

- Rewrite the grammar: use a bracketing keyword, e.g. Ada(**end if**), Algol68(**fi**), and Tiny

  if-stmt → **if** exp **then** stmt-sequence **end**
  | **if** exp **then** stmt-sequence **else** stmt-sequence **end**

- Use disambiguating rules: the most closely nested rule

  S → **if** E **then** S | **if** E **then** S **else** S | **other**

- 在能够控制和驾驭的情况下，可以使用二义性文法。
- 用最短的句子证明以下文法有二义性。

  1.S → S(S)S | ε

  2 . S → SS | (S) | ε

  3 . S → SS | (S) | ( )

  4.S → **i**S**e**S | **i**S | **i**

4.2.6 Verifying the Language Generated by a Grammar

- 证明文法G生成语言L的过程可以分为两个部分：
  - 证明G生成的每个串都在L中；反向证明L中的每个串都能由G生成。
- **Example 4.12** $S \rightarrow (S)S \mid \varepsilon$
  - Generates all strings of balanced parentheses, and only such strings.
  - 1. Show that every sentence derivable from S is balanced.
  - 2. Show that every balanced string is derivable from S.
  - Basis & Induction
- **Exercise 4.2.1** Consider the context-free grammar: $S \rightarrow SS+ \mid SS* \mid a$ and the string $aa+a*$.
  a) Give a leftmost derivation for the string.
  b) Give a rightmost derivation for the string.
  c) Give a parse tree for the string.
  ! d) Is the grammar ambiguous or unambiguous? Justify your answer.
  ! e) Describe the language generated by this grammar.

4.3 Writing a Grammar

4.3.2 Eliminating Ambiguity

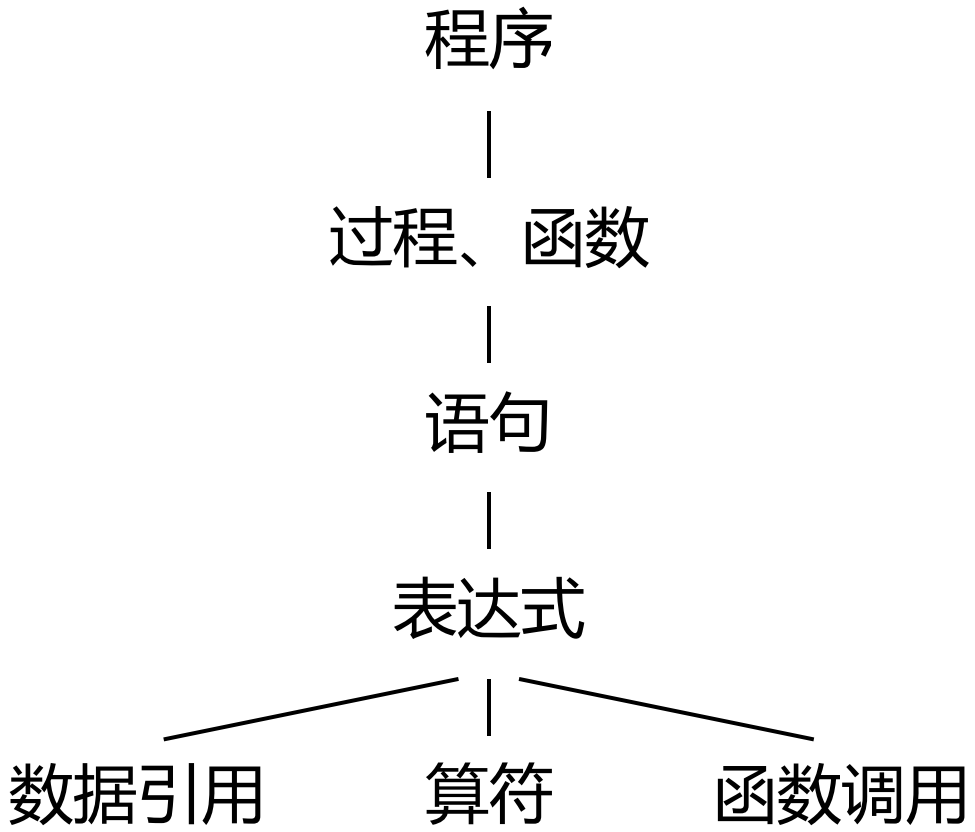
4.3.3 Elimination of Left Recursion

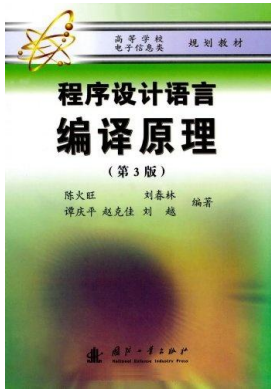
4.3.4 Left Factoring

Writing grammars for programming languages

Introduction to Formal Languages (4.2.7, 4.3.5, 4.3.1)

Hierarchy of Programming Languages





Grammar of Tiny ⟶ Tiny+

1. program → stmt-sequence
2. stmt-sequence → stmt-sequence; statement | statement
3. statement → if-stmt | repeat-stmt | assign-stmt | read-stmt | write-stmt
4. if-stmt → **if** exp **then** stmt-sequence **end** | **if** exp **then** stmt-sequence **else** stmt-sequence **end**
5. repeat-stmt → **repeat** stmt-sequence **until** exp
6. assign-stmt → **identifier** **:=** exp
7. read-stmt → **read identifier**
8. write-stmt → **write** exp
9. exp → simple-exp comparison-op simple-exp | simple-exp
10. comparison-op → **<** | **=** | **>** | **>=**
11. simple-exp → simple-exp addop term | term
12. addop → **+** | **-**
13. term → term mulop factor | factor
14. mulop → * | /
15. factor → **(**exp**)** | **number** | **identifier**

```
read x;
if 0<x then
      fact := 1;
      repeat
          fact := fact * x;
           x := x - 1
      until x = 0;
      write fact
end
```

- 变量声明语句：单变量/多变量
- 在定义变量的同时赋初值
- 数组
- 函数
- while语句和for语句
- 表达式中增加函数调用
- ……

Sequence of Statements

- 1. How statements are separated, and
- 2. whether they can contain empty statements.

stmt-seq → stmt ; stmt-seq | stmt stmt → **s**	{ s, s ; s , s ; s ; s , ... }
stmt-seq → stmt ; stmt-seq | ε stmt → **s**	{ ε , s; , s ; s; , s ; s ; s ; , ... }
stmt-seq → nonempty-ss | ε nonempty-ss → stmt; nonempty-ss | stmt stmt → **s**	{ ε , s , s ; s , s ; s ; s , ... }

C-

- 程序由声明列表组成。声明可以是变量声明或函数声明，顺序任意，至少有一个声明。
- 所有变量和函数必须在使用前声明。C-没有函数原型，因此声明和定义之间没有区别。
- 程序最后必须是一个形式为void main(void)的函数声明。

1. program → declaration-list
2. declaration-list → declaration-list declaration | declaration
3. declaration → var-declaration | fun-declaration
4. var-declaration → type-specifier **ID;** | type-specifier **ID [NUM];**
5. type-specifier → **int** | **void**
6. fun-declaration → type-specifier **ID (**params**)** compound-stmt
7. params → param-list | **void**
8. param-list→ param-list , param | param
9. param → type-specifier **ID** | type-specifier **ID [ ]**
10. compound-stmt → **{** local-declarations statement-list **}**
11. local-declarations → local-declarations var-declaration | empty

Sequence & Statements of C-

12. statement-list → statement-list statement | empty

How to separate statements?

13. statement → expression-stmt | compound-stmt | selection-stmt | iteration-stmt | return-stmt
14. expression-stmt → expression **;** | **;**
15. selection-stmt → **if (**expression**)** statement
    | **if (**expression**)** statement **else** statement
16. iteration-stmt → **while (** expression **)** statement
17. return-stmt → **return;** | **return** expression**;**

Grammar of C-

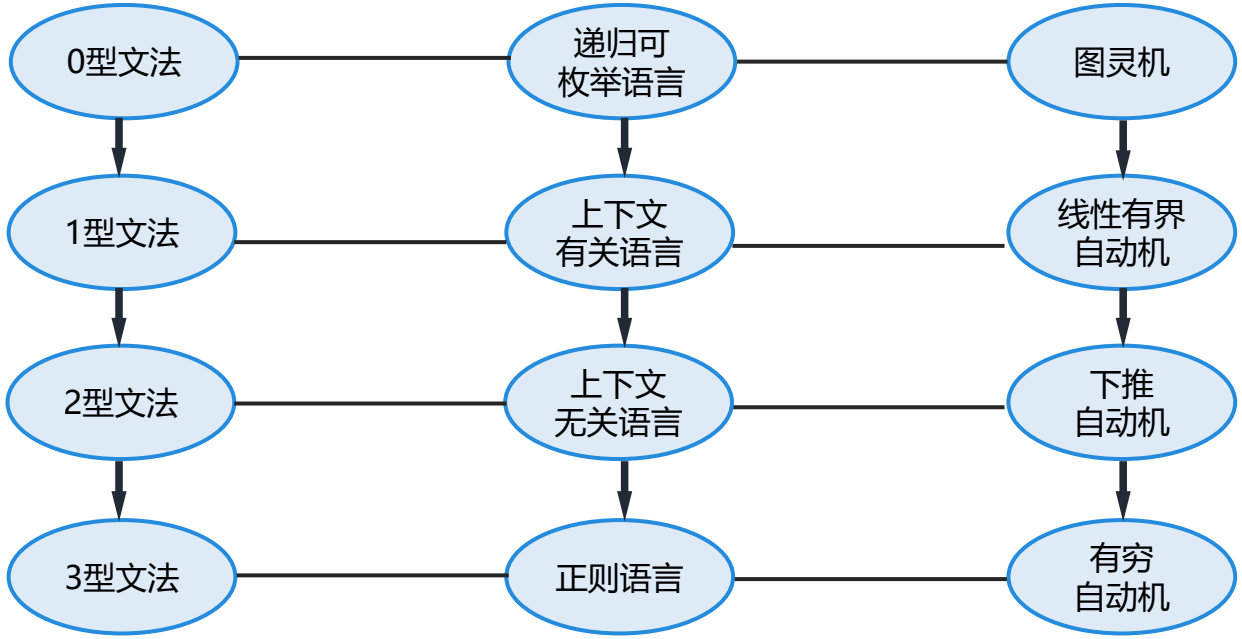
1. program → declaration-list
2. declaration-list → declaration-list declaration | declaration
3. declaration → var-declaration | fun-declaration
4. var-declaration → type-specifier **ID;** | type-specifier **ID [NUM];**
5. type-specifier → **int** | **void**
6. fun-declaration → type-specifier **ID (**params**)** compound-stmt
7. params → param-list | **void**
8. param-list→ param-list , param | param
9. param → type-specifier **ID** | type-specifier **ID [ ]**
10. compound-stmt → **{** local-declarations statement-list **}**
11. local-declarations → local-declarations var-declaration | empty
12. statement-list → statement-list statement | empty
13. statement → expression-stmt | compound-stmt | selection-stmt | iteration-stmt | return-stmt
14. expression-stmt → expression **;** | **;**
15. selection-stmt → **if (**expression**)** statement | **if (**expression**)** statement **else** statement
16. iteration-stmt → **while (** expression **)** statement
17. return-stmt → **return;** | **return** expression**;**
18. expression → var = expression | simple-expression
19. var → **ID** | **ID [** expression **]**
20. simple-expression → additive-expression relop additive-expression | additive-expression
21. relop → **<=** | **<** | **>** | **>=** | **==** | **!=**
22. additive-expression → additive-expression addop term | term
23. addop → **+** | **-**
24. term → term mulop factor | factor
25. mulop → ***** | */*
26. factor → **(** expression **)** | var | call | **NUM**
27. call → **ID (** args **)**
28. args → arg-list | empty
29. arg-list → arg-list **,** expression | expression

Introduction to Formal Languages

(4.2.7, 4.3.5, 4.3.1)

文法、形式语言与自动机

- 形式语言的起因：语言学家乔姆斯基想用一套形式化的方法来描述语言。
- 1956年，乔姆斯基从产生语言的角度研究语言：文法
- 1951-1956年间，克林在研究神经细胞的过程中建立了自动机，从识别的角度研究语言：自动机
- 1959年，乔姆斯基将他本人的研究成果与克林的研究成果结合了起来，证明了文法与自动机的等价性。
- 形式语言在自然语言研究中起步，在计算机科学中得到广泛应用。
  - 最初的应用：编译器；现在已广泛应用在人工智能、通信协议、通信软件等多个领域。
  - 在计算机理论科学方面，是计算复杂性理论、可计算理论等研究的基础。



Chomsky Hierarchy

- Chomsky于1956年建立形式语言体系，把文法分成四种类型：
  - 0型，
  - 1型，
  - 2型，
  - 3型。

$$G = < V_T, V_N, S, P >$$

- 四种文法的区别在于对产生式施加不同的限制。

文法表达能力：0>1>2>3

3型 —— 正规文法 —— 必须满足左线性或右线性文法
2型 —— 上下文无关文法 —— 必须要求产生式左边为非终结符
1型 —— 上下文有关文法 —— 产生式左部长度必须小于右部长度，但$S \rightarrow \varepsilon$是个例外
0型 —— 递归可枚举语言 —— 产生式左部和右部必须由终结符和非终结符组成

Type 3 Grammars

3型文法为正规文法，必须满足左线性文法或者右线性文法

根据个人理解，这里“线性”的意思是能够通过非终结符进一步进行推导。对于左线性，就是要求产生式右部第一个符号是非终结符（当然如果只有终结符就直接接终结符）

- Also called regular grammars, are equivalent to regular expressions.

left-linear grammar	right-linear grammar
$A \to B\alpha$ and $A \to \alpha$	$A \to \alpha B$ and $A \to \alpha$
Where $A,B \in V_N$, $\alpha \in V_T \cup \{\varepsilon\}$	or $\alpha \in V_T^*$

- Example

RE	Grammar	
a | b	$A \to a \mid b$	$A \to a \mid b$
ab	$B \to Ab$ $A \to a$	$A \to aB$ $B \to b$
a*	$A \to Aa \mid \varepsilon$	$A \to aA \mid \varepsilon$

From Regular Grammars to Regular Expressions

Grammar		RE	Example
left-linear grammar	$A \rightarrow a \mid b$	a | b	$S \rightarrow Aa \mid a$ $A \rightarrow Aa \mid a \mid Ab \mid b$
	$B \rightarrow Ab \quad A \rightarrow a$	ab	
	$B \rightarrow Bb \mid a$		
right-linear grammar	$A \rightarrow a \mid b$	a | b	$A_0 \rightarrow aA_0 \mid bA_0 \mid aA_1$ $A_1 \rightarrow bA_2$ $A_2 \rightarrow bA_3$ $A_3 \rightarrow \varepsilon$
	$A \rightarrow aB \quad B \rightarrow b$	ab	
	$A \rightarrow aA \mid b$		

From Regular Expressions to Regular Grammars

RE	left-linear grammar	right-linear grammar
a|b	$A \rightarrow a \mid b$	$A \rightarrow a \mid b$
ab	$B \rightarrow Ab \quad A \rightarrow a \quad (B \rightarrow ab)$	$A \rightarrow aB \quad B \rightarrow b \quad (A \rightarrow ab)$
ab*	B->Bb|a	S->aB B->bB|ε
a*b	S->Ab A->Aa|ε	A->aA|b
ab*c	S->Bc B->Bb|a	S->aB B->bB|c
a*b*	S->Sb|A A->Aa|ε	S->aS|B B->bB|ε

Type 2 Grammars

2型文法要求产生式的左部必须是一个非终结符，一般形式为A->β

- Also called context free grammars.

$$A \to \beta, \quad \text{where } A \in V_N, \ \beta \in (V_T \cup V_N)^*$$

- For example, grammar of Tiny / C–

- $L_1 = \{ a^n b^n \mid n \geqslant 1 \}$
- $L_2 = \{ a^n b^n a^m b^m \mid n,m \geqslant 0 \}$
- $L_3 = \{ a^n b^{2(n+1)} c \mid n \geqslant 0 \}$
- $L_4 = \{ a^m b^n \mid m \geqslant n, n \geqslant 0 \}$
- $L_5 = \{ a^m b^n \mid 1 \leqslant n \leqslant m \leqslant 2n \}$
- $L_6 = \{ ww^R \mid w \in (0|1)^* \}$, where w^R is the reverse order of w.
- $L_7 : \Sigma = \{a, b\}$, the set of strings with the same number of a's and b's.
- $L_8 = \{ a^n b^n c^n \mid n \geqslant 1 \}$

L1:S->aSb | ab
L2:S->AB
A->aAb | ε
B->aBb | ε

L3: S->Abbc
A->aAbb | ε
L4: S->AB
A->aA | ε
B->bB | ε

L5: S->aSb | aaSb | ε
L6: S->1S1 | 0S0 | 0 | 1 | ε
L8: S->aS | bSc | ε

L7: S -> aSbS | bSaS | ε

L4: S -> aSb | A
A -> aA | ε

Type 1 Grammars

1型文法产生式的左部长度必须小于等于右部长度，但需要注意S->ε是一个例外，也称之为上下文有关文法。

- Also called context sensitive grammars.

$\alpha \rightarrow \beta$, where $\alpha \in (V_T \cup V_N)^*$ and include at least one nonterminal,

$\beta \in (V_T \cup V_N)^*$, and $|\alpha| \leqslant |\beta|$, except $S \rightarrow \varepsilon$.

$S \rightarrow aBC \mid aSBC$

$CB \rightarrow BC$

$aB \rightarrow ab$

$bB \rightarrow bb$

$bC \rightarrow bc$

$cC \rightarrow cc$

$A \rightarrow abc \mid aBbc$

$Bb \rightarrow bB$

$Bc \rightarrow Cbcc$

$bC \rightarrow Cb$

$aC \rightarrow aaB$

$aC \rightarrow aa$

Type 0 Grammars

- Also called phase grammars, or unrestricted grammars.

  $\alpha \rightarrow \beta$, where $\alpha \in (V_T \cup V_N)^*$ and include at least one nonterminal, $\beta \in (V_T \cup V_N)^*$

- **Example 4.25** Identifiers are declared before they are used in a program.

  $L = \{ \alpha c \alpha \mid \alpha \in (a|b)^* \}$

- **Example 4.26**

  $L = \{ a^n b^m c^n d^m \mid n,m \geqslant 1 \}$

0型文法的产生式的左部或者右部必须由终结符或非终结符构成

$S \rightarrow aBC \mid aSBC$

$CB \rightarrow BC$

$aB \rightarrow ab$

$bB \rightarrow bb$

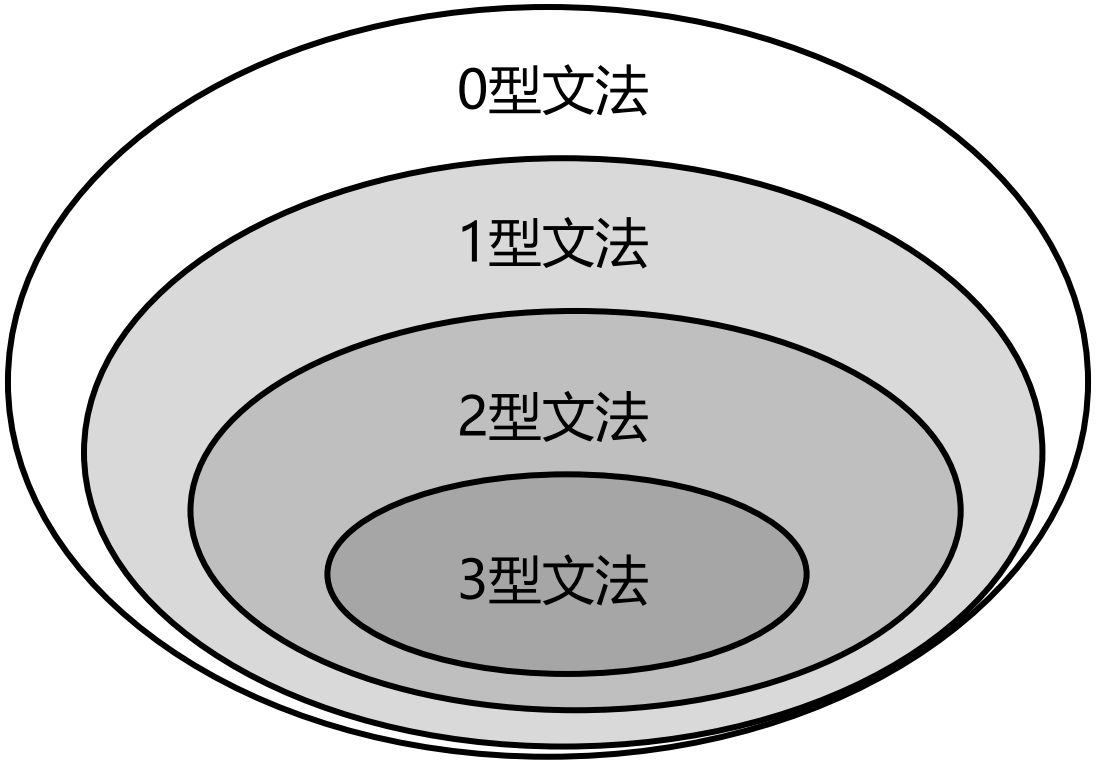
$bB \rightarrow b$

$bC \rightarrow bc$

$cC \rightarrow cc$

$cC \rightarrow c$

Power of Description



程序设计语言不是上下文无关语言，甚至不是上下文有关语言。

上下文无关文法有足够的能力描述大多数程序设计语言的语法结构。

3.1.1 Lexical Analysis Versus Parsing

“ Why the analysis portion of a compiler is normally separated into lexical analysis and syntax analysis (parsing) phases? ”

1. 简化编译器的设计
2. 提高编译器的效率
3. 增强编译器的可移植性

4.3.1 Lexical Versus Syntactic Analysis

“ Why use regular expressions to define the lexical syntax of a language? ”

1. 将一个语言的语法结构分为词法和非词法两部分可以很方便地将编译器前端模块化，将前端分解为两个大小适中的构件。
2. 一个语言的词法规则通常很简单，不需要使用像文法这样概念强大的表示方法来描述这些规则。
3. 和文法相比，正则表达式提供了更简洁更易于理解的词法单元表示方法。
4. 根据正则表达式自动构造得到的词法分析器的效率要高于根据任何文法自动构造得到的词法分析器的效率。

- 并不存在一个严格的指导方针来规定哪些内容应该放到（和语法规则相对的）词法规则中。
- 正则表达式不能匹配任意深度的嵌套结构。在很长的时间内，这是放之四海而皆准的规则。
- 但是现在，Perl、.NET和PCRE/PHP都提供了解决办法。
  - Perl：动态正则表达式

4.4 Top-Down Parsing

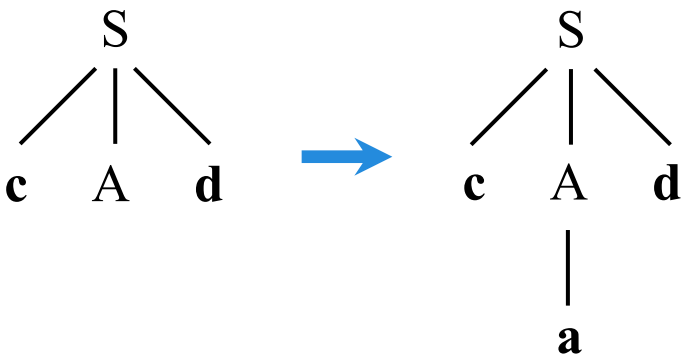
- 从根节点开始，向下构造分析树，直到创建每个叶节点。
- 从文法的开始符号出发，根据文法规则推导出给定句子，这个过程和最左推导对应。

Deterministic vs. Nondeterministic

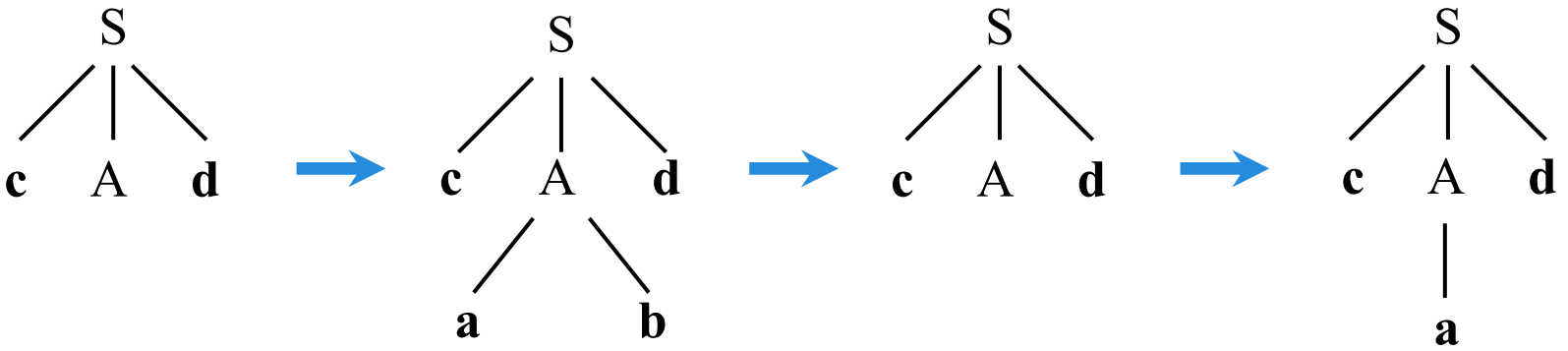
- Example 4.29　Consider the following grammar for the string **cad**.

$S \rightarrow \mathbf{c}A\mathbf{d}$

$A \rightarrow \mathbf{ab} \mid \mathbf{a}$



- 通过向前看1个或多个token，准确判断下一步选择哪一个产生式（预测分析）。



- 是一种穷举试探的过程，是反复使用不同产生式谋求匹配输入串的过程。试探发生回溯。

Two forms of Top-Down Parsers

Parsers	Characteristics	Methods
预解析器 Predictive	• Deterministic top-down parsing	递归下降分析 Recursive-descent parsing LL(1)解析器 LL(1) parsing
回溯解析器 Backtracking	• Nondeterministic top-down parsing • more powerful than predictive parsers but much slower, unsuitable for practical compilers.	递归下降分析 Recursive-descent parsing

4.4.1 Recursive-Descent Parsing

CONTENTS

Basic Method

- 基本思想
  - 把非终结符A的文法规则看作对A的识别过程的定义。
  - 右侧符号串指定了识别过程的代码结构。
    - 有两个或多个候选式的时候，用 if 或 case 结构来处理。
    - 遇到终结符就匹配，遇到非终结符就调用。

```
procedure match(expectedToken)
begin
  if token=expectedToken then
    getToken;
  else
    error;
  end if;
end match;
```

$$factor \rightarrow (exp) \mid \textbf{\textit{number}}$$

```
procedure factor
begin
  case token of
    ( :
      match(();
      exp;
      match());
    number :
      match(number);
    else error;
  end case;
end factor;
```

Problems & Solutions

- 如果文法中存在以下两个问题，就不能进行确定的自顶向下分析：
  - 左递归
  - 左因子 → 回溯

$$exp \rightarrow exp\ addop\ term \mid term$$

$$term \rightarrow term\ mulop\ factor \mid factor$$

$$factor \rightarrow \mathbf{(}exp\mathbf{)} \mid \mathbf{number}$$

$$if\text{-}stmt \rightarrow \mathbf{if}\ \mathbf{(}exp\mathbf{)}\ statement \mid \mathbf{if}\ \mathbf{(}exp\mathbf{)}\ statement\ \mathbf{else}\ statement$$

- 要进行确定的自顶向下分析
  - 去除左递归
  - 消除回溯

} 改写文法（EBNF / BNF）

Rewrite Grammars Using EBNF

{}表示重复，类似于*； []表示可选，要门不出现要么只出现一次

- EBNF，扩展的巴科斯范式，为递归下降分析程序而设计，不能用于LL(1)分析。
- Uses { . . . } to express repetition.
- BNF: $A \rightarrow A\ \alpha \mid \beta$ EBNF: $A \rightarrow \beta\ \{\alpha\}$

exp → *exp addop term* | *term*

exp → *term* {*addop term*}

```
procedure exp;
begin
   term;
   while token = + or token = - do
      match(token);
      term;
   end while;
end exp;
```

addop → + | -

term → *term mulop factor* | *factor*

- Uses [...] to express optional construct.

if-stmt → **if** (*exp*) *statement*

| **if** (*exp*) *statement* **else** *statement*

if-stmt → **if** (*exp*) *statement* [**else** *statement*]

```
procedure ifstmt;
begin
   match(if);
   match(();
   exp;
   match());
   statement;
   if token = else then
      match(else);   // 精准地对应了最近嵌套原则
      statement;
   end if;
end ifstmt;
```

Example: A Working Simple Calculator

- *exp* → *term* { *addop term* } (procedure vs. function)

```
procedure exp;
begin
  term;
  while token=+ or token=- do
    match(token);
    term;
  end while;
end exp;
```

```
function exp: integer;
var temp: integer;
begin
  temp:=term;
  while token=+ or token=- do
      case token of
      + : match(+); temp:=temp+term;
      - : match(-); temp:=temp-term;
    end case;
  end while;
  return temp;
end exp;
```

BNF: S -> +S'

S' -> aAS' | ε

EBNF: S -> +{aA}

```
void parseS() {
   match('+');
   while (getNextchar == 'a') {
      match('a');
      parseA();
   }
}
```

- Calculator.c
- Right Associativity

exp → *term addop exp* | *term*

exp → { *term addop* } *term*

exp → *term* [*addop exp*]

Consider the grammar:

S → S a A | +

A → (a) * b | (a)

BNF: A -> (a)B

B -> *b | ε

EBNF: A -> (a)[*b]

```
void parseA() {
   match('(');
   match('a');
   match(')');
   if (getNextChar() == '*') {
      match('*');
      match('b');
   }
}
```

Abstract Syntax Trees

- 在语法树中去掉那些对翻译不必要的信息，从而获得更有效的源程序中间表示。
- 经过这种变换之后得到的语法树称为抽象语法树（AST）。

exp → *exp addop term* | *term*
addop → + | -
term → *term mulop factor* | *factor*
mulop → *

factor → (*exp*) | ***number*** | **id**

E → E+T | E-T | T
T → T*F | F
F → (E) | **n** | **id**

x*(2+3)

statement → *if-stmt* | *repeat-stmt* | *assign-stmt* | *read-stmt* | *write-stmt*

stmt-sequence → *stmt-sequence*; *statement* | *statement*
statement → **s**

s; s; s

Data type declaration

```
typedef enum {OpK,ConstK,IDK} ExpKind;
typedef struct streenode
{
    ExpKind kind;
    union {TokenType op; int val; char* name;} attr;
    struct streenode *lchild,*rchild;
} STreeNode;
STreeNode *SyntaxTree;
```

```
typedef enum {IfK,RepeatK,AssignK,ReadK,WriteK} StmtKind;
typedef struct streenode
{
  StmtKind kind;
  struct streenode *test,*then,*else;
} STreeNode;
STreeNode *SyntaxTree;
```

- Globals.h

Code

exp → *term* { *addop term* }

```
function exp: syntaxTree;
var temp,newtemp: syntaxTree;
begin
  temp:=term;
  while token=+ or token=- do
    newtemp:=makeOpNode(token);
    match(token);
    leftChild(newtemp):= temp;
    rightChild(newtemp):= term;
    temp:=newtemp;
  end while;
  return temp;
end exp;
```

1+2*3

factor → **(***exp***)** | ***number***

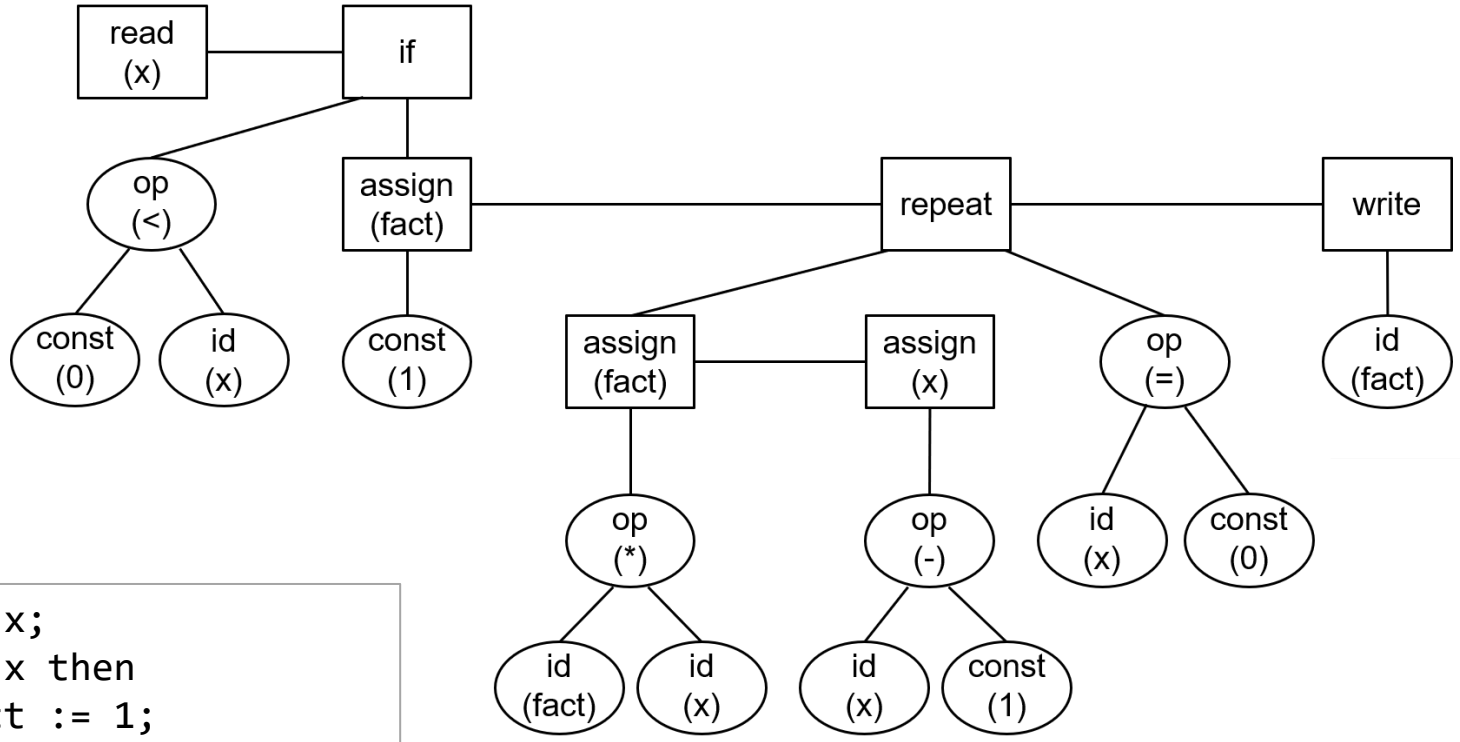
```
function factor: syntaxTree;
var temp: syntaxTree;
begin
  case token of
    (:
      begin: match('(');
             temp := exp();
             match(')');
      end;
    number:
      begin: temp := makeLeafNode(token);
             match(number);
      end;
    else error;
  end case;
  return temp;
end factor;
```

if-stmt → **if (***exp***)** *statement* [**else** *statement*]

```
function ifstatement: syntaxTree;
var temp, expNode, thenNode, elseNode:syntaxTree;
begin
    temp:= newStmtNode(IfK);
    match('if');
    match('(');
    expNode := exp();
    match(')');
    thenNode := statement();

    if token = 'else' then
       match('else');
       elseNode := statement();
    else
       elseNode := NULL;

    leftChild(temp) := expNode;
    middleChild(temp) := thenNodo;
    rightChile(temp) := elseNode;
     return temp;
 end ifstatement;
```

- [TreeNode * simple_exp(void);]

Syntax Tree for the Sample Program




```
read x;
if 0<x then
  fact := 1;
  repeat
    fact := fact * x;
    x := x - 1
  until x = 0;
  write fact
end
```

Read: x
If
    Op: <
        const: 0
        Id: x
    Assign to: fact
        const: 1
    Repeat
        Assign to: fact
            Op: *
                Id: fact
                Id: x
        Assign to: x
            Op: -
                Id: x
                const: 1
        Op: =
            Id: x
            const: 0
    Write
        Id: fact

A Recursive-Descent Parser for TINY

program → *stmt-sequence*

stmt-sequence → *stmt-sequence* **;** *statement* | *statement*

statement → *if-stmt* | *repeat-stmt* | *assign-stmt* | *read-stmt* | *write-stmt*

if-stmt → **if** *exp* **then** *stmt-sequence* **end**

| **if** *exp* **then** *stmt-sequence* **else** *stmt-sequence* **end**

repeat-stmt → **repeat** *stmt-sequence* **until** *exp*

assign-stmt → ***identifier*** **:=** *exp*

read-stmt → **read** ***identifier***

write-stmt → **write** *exp*

exp → *simple-exp comparison-op simple-exp* | *simple-exp*

comparison-op → < | =

simple-exp → *simple-exp addop term* | *term*

addop → + | -

term → *term mulop factor* | *factor*

mulop → * | /

factor → (*exp*) | ***number*** | ***identifier***

program → *stmt-sequence*

stmt-sequence → *statement* { **;** *statement* }

statement → *if-stmt* | *repeat-stmt* | *assign-stmt* | *read-stmt* | *write-stmt*

if-stmt → **if** *exp* **then** *stmt-sequence* [**else** *stmt-sequence*] **end**

repeat-stmt → **repeat** *stmt-sequence* **until** *exp*

assign-stmt → ***identifier*** **:=** *exp*

read-stmt → **read** ***identifier***

write-stmt → **write** *exp*

exp → *simple-exp* [*comparison-op simple-exp*]

comparison-op → < | =

simple-exp → *term* { *addop term* }

addop → + | -

term → *factor* { *mulop factor* }

mulop → * | /

factor → (*exp*) | ***number*** | ***identifier***

- The parse.c consists of 11 mutually recursive procedure that correspond directly to the EBNF grammar.

Top-down Parsing

- Problems
  - Top-down parsing methods cannot handle left-recursive grammars.
  - Deterministic top-down parsing cannot handle left-factor grammars.
- Solutions
  - Elimination of left recursion
  - Left factoring

4.3.3 Elimination of Left Recursion

直接左递归	$A \to A\alpha \mid \beta,$ where $\alpha, \beta \in (V_T \cup V_N)^*$, β does not begin with A.	$A \to \beta A'$ $A' \to \alpha A' \mid \varepsilon$
	$E \to E + T \mid T$ $T \to T * F \mid F$	$E \to TE'$ $\quad T \to FT'$ $E' \to +TE' \mid \varepsilon$ $\quad T' \to *FT' \mid \varepsilon$
一般的 直接左递归	$A \to A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_m,$ where none of $\beta_1, \ldots, \beta_m$ begin with A.	$A \to \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_m A'$ $A' \to \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_n A' \mid \varepsilon$
	$E \to E + T \mid E - T \mid T$	$E \to TE'$ $E' \to +TE' \mid -TE' \mid \varepsilon$
间接左递归	$S \to Qc \mid c$ $Q \to Rb \mid b$ $R \to Sa \mid a$ $S \Rightarrow Qc \Rightarrow Rbc \Rightarrow Sabc$	通过产生式中的非终结符置换， 将间接左递归改为直接左递归。

- $seq \to seq; s \mid s$ $\qquad$ $seq \to s ; seq \mid s$?

4.3.4 Left Factoring

- two A-productions

$A \to \alpha\beta \mid \alpha\gamma$	$A \to \alpha A'$ $A' \to \beta \mid \gamma$
if-stmt → **if** (E) S | **if** (E) S **else** S	*if-stmt* → **if** (E) S *else-part* *else-part* → **else** S | ε

- more A-productions

$A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \ldots \mid \alpha\beta_n$
$A \to \alpha A'$ $A' \to \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$ 反复提取左因子

- Example 4.22

$S \to \boldsymbol{i}\ E\ \boldsymbol{t}\ S \mid \boldsymbol{i}\ E\ \boldsymbol{t}\ S\ \boldsymbol{e}\ S \mid \boldsymbol{a}$

$E \to \boldsymbol{b}$ (4.23)

$S \to \boldsymbol{i}\ E\ \boldsymbol{t}\ S\ S' \mid \boldsymbol{a}$

$S' \to e\ S \mid \varepsilon$

$E \to \boldsymbol{b}$ (4.24)

- Consider the following grammar:

$S \to (S)S \mid (L)$

$L \to \mathbf{i} \mid L, \mathbf{i}$

S -> (A
A -> S)S | L)
L -> iL'
L' -> ,iL'

$S \to S\ a\ A \mid +$

$A \to (\ a\ )\ *\ b \mid (\ a\ )$

4.4.2 FIRST and FOLLOW

FIRST

- Define FIRST(α), where α is any string of grammar symbols, to be the set of terminals that begin strings derived from α.

  FIRST(α)是指从α推导得到的所有串中位于串首的终结符所组成的集合。

$$\mathrm{FIRST}(\alpha) = \{\, a \mid \alpha \Rightarrow^* a\beta,\ a \in V_T,\ \beta \in (V_T \cup V_N)^* \,\}$$

$$\text{If } \alpha \Rightarrow^* \varepsilon, \text{ then } \varepsilon \in \mathrm{FIRST}(\alpha).$$

Computing FIRST(X)

- 对文法符号 $X \in V_T \cup V_N$，计算FIRST(X)。**连续使用以下规则，直到FIRST集合不再增加元素**：
  1. If $X \in V_T$, then $FIRST(X) = \{X\}$.
  2. If $X \in V_N$ and $X \to \varepsilon$ is a production, then add ε to $FIRST(X)$.
  3. If $X \in V_N$ and $X \to Y_1 Y_2 \ldots Y_k$ $(k \geq 1)$ is a production,
     ① add $FIRST(Y_1)-\{\varepsilon\}$ to $FIRST(X)$;
     ② if for some $i<k$, ε is in all of $FIRST(Y_1)$, …, $FIRST(Y_i)$, add $FIRST(Y_{i+1})-\{\varepsilon\}$ to $FIRST(X)$;
     ③ if ε is in all of $FIRST(Y_1)...FIRST(Y_k)$, add ε to $FIRST(X)$.
- **Example** Compute FIRST for all symbols X of the following grammar.

$S \to ABC$

$A \to \varepsilon \mid a$

$B \to bc \mid Ad \mid \varepsilon$

$C \to ABc \mid \varepsilon$

- FIRST(a)
- FIRST(b)
- FIRST(c)
- FIRST(d)

- FIRST(S)={a, b, d, c, ε}
- FIRST(A)={a, ε}
- FIRST(B)={a, b, d, ε}
- FIRST(C)={a, b, c, d, ε}

Computing FIRST(α)

- 对符号串 $\alpha = X_1X_2\ldots X_n$ ，计算FIRST(α)。**连续使用以下规则，直到FIRST集合不再增加元素**：
  1. $FIRST(\alpha) = FIRST(X_1) - \{\varepsilon\}$;
  2. If for any $1 \leqslant j \leqslant i-1$, $\varepsilon \in FIRST(X_j)$, then add $FIRST(X_i) - \{\varepsilon\}$ to $FIRST(\alpha)$;

     If for all i $(1 \leqslant i \leqslant n)$, $\varepsilon \in FIRST(X_i)$, then add ε to $FIRST(\alpha)$.

- **Example 4.30 - I**

Production	Pass1	Pass2	Pass3
E → TE′			
E′ → +TE′ | ε	{ +, ε }		
T → FT′		{ (, i }	
T′ → *FT′ | ε	{ *, ε }		
F → (E) | i	{ (, i }		

1.如果产生式右边是非终结符，则直接加入First集
2.如果是产生式右边非终结符则将该非终结符的First集加入左边非终结符的First集合。若进一步的，产生式右边的非终结符的First集中包含ε则将下一个非终结符的First集加入左边非终结符的First集合，直至出现非终结符或不包含ε

Production	FIRST
S → ABe	{a, b, d, e, ε}
A → a | ε	{a, ε}
B → bSc | dA | ε	{b, d, ε}

Why Compute FIRST ?

- For $A \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$, for all i and j, $1 \leqslant i, j \leqslant n$, $i \neq j$,

  if $FIRST(\alpha_i) \cap FIRST(\alpha_j) = \phi$, and $a \in FIRST(\alpha_i)$, then select $A \to \alpha_i$.

$A \to \alpha \mid \beta$	$F \to (E) \mid \mathbf{i}$	$FIRST(\alpha) \cap FIRST(\beta) = \phi$
	if-stmt $\to$ **if** (E) S | **if** (E) S **else** S	$FIRST(\alpha) \cap FIRST(\beta) \neq \phi$
$A \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$	*statement* $\to$ *if-stmt* | *repeat-stmt* | *assign-stmt* | *read-stmt* | *write-stmt*	

- 如果文法满足这样的条件，是否就一定能进行确定的自顶向下分析?
- For an ε-production: $A \to \varepsilon$, or $\varepsilon \in FIRST(\alpha_i)$
  - 是否一定可以选择 $A \to \varepsilon$ 继续推导?
  - 需要用FOLLOW集合来判断哪些终结符可以合法地跟随在A之后。

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid i$

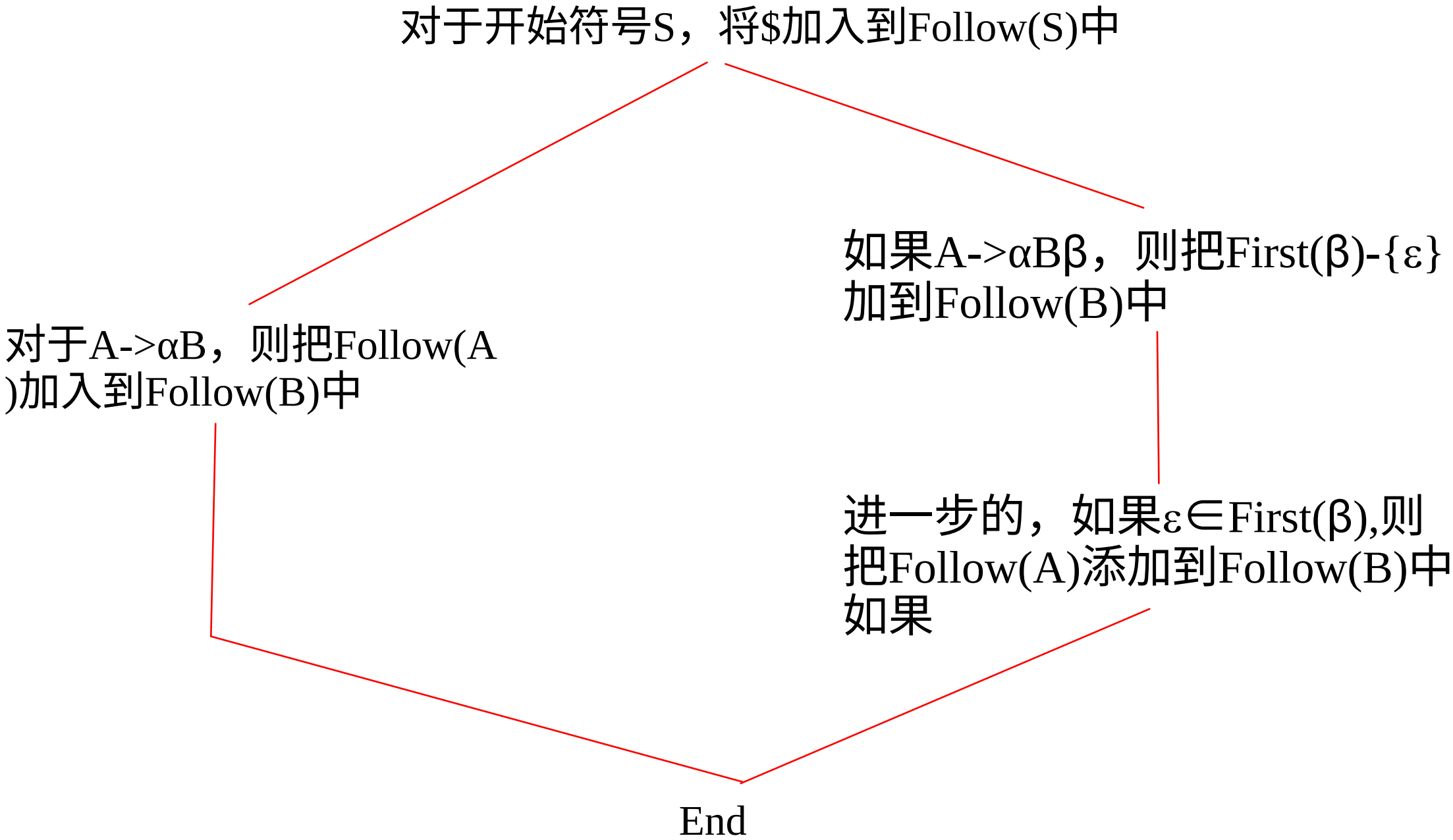
FOLLOW

- Define FOLLOW (A), $A \in V_N$,
  - to be the set of terminals a that can appear immediately to the right of A in some sentential form;
  - In addition, if A can be the rightmost symbol in some sentential form, then $ is in FOLLOW(A). $ 是整个输入串的结束标记。

FOLLOW(A)是文法G的某些句型中紧随在A之后出现的终结符 或 $。

$$\text{FOLLOW}(A) = \{ a \mid S \Rightarrow^* \alpha A a \beta,\ a \in V_T \}$$

$$\text{If } S \Rightarrow^* \ldots A, \text{ then } \$ \in \text{FOLLOW}(A).$$


对于开始符号S，将$加入到Follow(S)中
如果A->αBβ，则把First(β)-{ε}加到Follow(B)中
对于A->αB，则把Follow(A)加入到Follow(B)中
进一步的，如果ε∈First(β),则把Follow(A)添加到Follow(B)中
如果
End

Example

1. 对文法的开始符号S，$∈FOLLOW(S)；
2. 如果 A→αBβ 是一个产生式，则把 FIRST(β)-{ε} 添加到 FOLLOW(B) 中；
3. 如果 ① A→αB 是一个产生式，
   则把FOLLOW(A)添加到FOLLOW(B)中。

Production	FIRST	FOLLOW
E → E+T | T	{ (, i }	= { $ } ∪ FIRST(+T) ∪ FIRST( )) = { $, +,) }
T → T*F | F	{ (, i }	= FOLLOW(E) ∪ FOLLOW(E) ∪ FIRST(*F) = { $, +,), * }
F → (E) | i	{ (, i }	= FOLLOW(T) ∪ FOLLOW(T) = { $, +,), * }

Example 4.30 - II

1. 对文法的开始符号S，$\$\in FOLLOW(S)$；
2. 如果 $A\to\alpha B\beta$ 是一个产生式，则把 $FIRST(\beta)-\{\varepsilon\}$ 添加到 $FOLLOW(B)$ 中；
3. 如果 ① $A\to\alpha B$ 是一个产生式，或 ② $A\to\alpha B\beta$是一个产生式，且$\varepsilon\in FIRST(\beta)$，则把FOLLOW(A)添加到FOLLOW(B)中。

Production	FIRST	FOLLOW
$E \to TE'$	{ (, i }	= { \$ } ∪ FIRST( )) = { \$,) }
$E' \to +TE' \mid \varepsilon$	{ +, ε }	= FOLLOW(E) ∪ FOLLOW(E′) = { \$,) }
$T \to FT'$	{ (, i }	= (FIRST(E′)-{ε}) ∪ (FIRST(E′)-{ε}) ∪ FOLLOW(E) ∪ FOLLOW(E′) = { \$,), + }
$T' \to *FT' \mid \varepsilon$	{ *, ε }	= FOLLOW(T) ∪ FOLLOW(T′) = { \$,), + }
$F \to (E) \mid i$	{ (, i }	= (FIRST(T′)-{ε}) ∪ (FIRST(T′)-{ε}) ∪ FOLLOW(T) ∪ FOLLOW(T′) = { \$,), +,* }

Computing FOLLOW(A)

- 对文法G的每个非终结符，计算其FOLLOW集合。

  连续使用以下规则，直到每个FOLLOW集合不再增加元素：

  1. 对文法的开始符号S，$\$\in FOLLOW(S)$；
  2. 如果$A\rightarrow\alpha B\beta$是一个产生式，则把$FIRST(\beta)-\{\varepsilon\}$添加到FOLLOW(B)中；
  3. 如果①$A\rightarrow\alpha B$是一个产生式，或②$A\rightarrow\alpha B\beta$是一个产生式，且$\varepsilon\in FIRST(\beta)$，则把FOLLOW(A)添加到FOLLOW(B)中。

- Notes: 1. FOLLOW sets are defined only for nonterminal. 2. The ε is never an element of a FOLLOW set.

Production	FIRST	FOLLOW
$S \rightarrow ABe$	{ a, b, d, e }	{$, c}
$A \rightarrow a \mid \varepsilon$	{ a, ε }	{b, d, $, c, e}
$B \rightarrow bSc \mid dA \mid \varepsilon$	{ b, d , ε }	{e}

Production	FIRST	FOLLOW
$S \rightarrow iEtSS' \mid a$		
$S' \rightarrow eS \mid \varepsilon$		
$E \rightarrow b$		

Follow(S)={$}+First(c)
Follow(A)=First(B)-ε+Follow(S)+Follow(B)
Follow(B)=First(e)

Why Compute FIRST and FOLLOW

- 1. For $A \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$, for all ***i*** and ***j***, $1 \leqslant i, j \leqslant n$, $i \neq j$,

  if $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \phi$, and $a \in \text{FIRST}(\alpha_i)$, then select $A \to \alpha_i$.

- 2. For an ε-production: $A \to \varepsilon$, or $\varepsilon \in \text{FIRST}(\alpha_i)$,
  - it may be necessary to know what tokens legally coming after A.
  - This set of such tokens is called the FOLLOW set of A.
- 3. To detect the errors as early as possible.
  - Such as “)3-2)”, the parser will descend from *exp* to *term* to *factor* before an error is reported.

```
procedure exp;
begin
  term;
  while token=+ or token=- do
     match(token); term;
  end while;
end exp;
```

```
procedure factor
begin
  case token of
          ( : match(); exp; match());
    number : match(number);
      else   error;
  end case;
end factor;
```

4.4.3 LL(1) Grammars

- LL(1) & LL(k)
  - The 1st "L" stands for scanning the input from left to right,
  - the 2nd "L" for producing a leftmost derivation, and
  - the "1" for using one input symbol of lookahead to predict.
  - k是指往前看k个符号。LL(k)分析的使用并不普遍，因为往前多看若干个token，在计算和使用上变得更复杂，但功能上并没有增加多少。

Algorithm 4.31 (Construction of Parsing Table)

- The algorithm 4.31 collects the information from FIRST and FOLLOW sets into a table M[A, a].
- For each production $A \rightarrow \alpha$, do the following:
  1. For each $a \in FIRST(\alpha)$, add $A \rightarrow \alpha$ to M[A, a].
  2. If $\varepsilon \in FIRST(\alpha)$, for each $b \in FOLLOW(A)$ (a terminal or $), add $A \rightarrow \alpha$ to M[A, b].

After performing the above, an empty entry means **error**.

Production	FIRST	FOLLOW
$E \rightarrow TE'$	{ (, i }	{), $ }
$E' \rightarrow +TE' \mid \varepsilon$	{ +, ε }	{), $ }
$T \rightarrow FT'$	{ (, i }	{ +,), $ }
$T' \rightarrow *FT' \mid \varepsilon$	{ *, ε }	{ +,), $ }
$F \rightarrow (E) \mid i$	{ (, i }	{ +, *,), $ }

	i	+	*	(	)	$
E						
E′						
T						
T′						
F						

- $S \rightarrow (S)S \mid \varepsilon$

Parsing Table vs. LL(1) Grammar

- 算法4.31可以应用于任意文法，生成该文法的预测分析表。
- 如果该文法是LL(1)文法，则预测分析表中每个表项最多只有一个产生式。
- 如果预测分析表中每个表项最多只有一个产生式，则该文法是LL(1)文法。

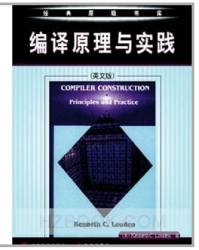
NON-TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to \mathbf{id}$			$F \to (E)$		

Figure 4.17: Parsing table M for Example 4.32

LL(1) Grammars

- A grammar G is LL(l) if and only if whenever $A \to \alpha \mid \beta$ are two distinct productions of G, the following conditions hold:
  1. For no terminal *a* do both α and β derive strings beginning with *a*.
  2. At most one of α and β can derive the empty string.
  3. If $\beta \Rightarrow^* \varepsilon$, then α does not derive any string beginning with a terminal in FOLLOW(A). Likewise, if $\alpha \Rightarrow^* \varepsilon$, then β does not derive any string beginning with a terminal in FOLLOW(A).

- A grammar G is LL(l) if and only if the following conditions hold:
  1. For every $A \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$, $FIRST(\alpha_i) \cap FIRST(\alpha_j) = \phi$, where $1 \leqslant \boldsymbol{i}, \boldsymbol{j} \leqslant \boldsymbol{n}, \boldsymbol{i} \neq \boldsymbol{j}$.
  2. For every $A \in V_N$, if $\varepsilon \in FIRST(A)$, then $FIRST(A) \cap FOLLOW(A) = \phi$.



- If a grammar is LL(1), deterministic top-down parsing can be performed.

Production	FIRST	FOLLOW
$E \to TE'$	{ (, i }	{ $,) }
$E' \to +TE' \mid \varepsilon$	{ +, ε }	{$,) }
$T \to FT'$	{ (, i }	{ +, $,) }
$T' \to *FT' \mid \varepsilon$	{ *, ε }	{ +, $,) }
$F \to (E) \mid i$	{ (, i }	{ *, +, $,) }

Production	FIRST	FOLLOW
$E \to E+T \mid T$	{ (, i }	
$T \to T*F \mid F$	{ (, i }	
$F \to (E) \mid i$	{ (, i }	

$S \to S(S)S \mid \varepsilon$

Parsing Table vs. Non-LL(1) Grammar

- 某些文法的预测分析表中，有的表项包含不只一个产生式，例如左递归文法或二义性文法。

Production	FIRST
E → E+T | T	{ (, i }
T → T*F | F	{ (, i }
F → (E) | i	{ (, i }

	i	+	*	(	)	$
E						
T						
F						

- **Example 4.33**

  S → iEtS | iEtSeS | a

  E → b

  - 该文法是不是LL(1)文法?
  - 并非所有的文法都能改写为LL(1)文法。
  - 最近嵌套原则

Production	FIRST	FOLLOW	
S → iEtSS′ | a			
S′ → eS | ε			FIRST(S′)∩FOLLOW(S′) ≠ ϕ
E → b			

	a	b	e	i	t	$
S	S → a			S → iEtSS′		
S′			S′→ eS S′→ ε			S′ → ε
E		E → b				

4.4.4 Nonrecursive Predictive Parsing

- Maintain a stack explicitly, rather than implicitly via recursive calls.

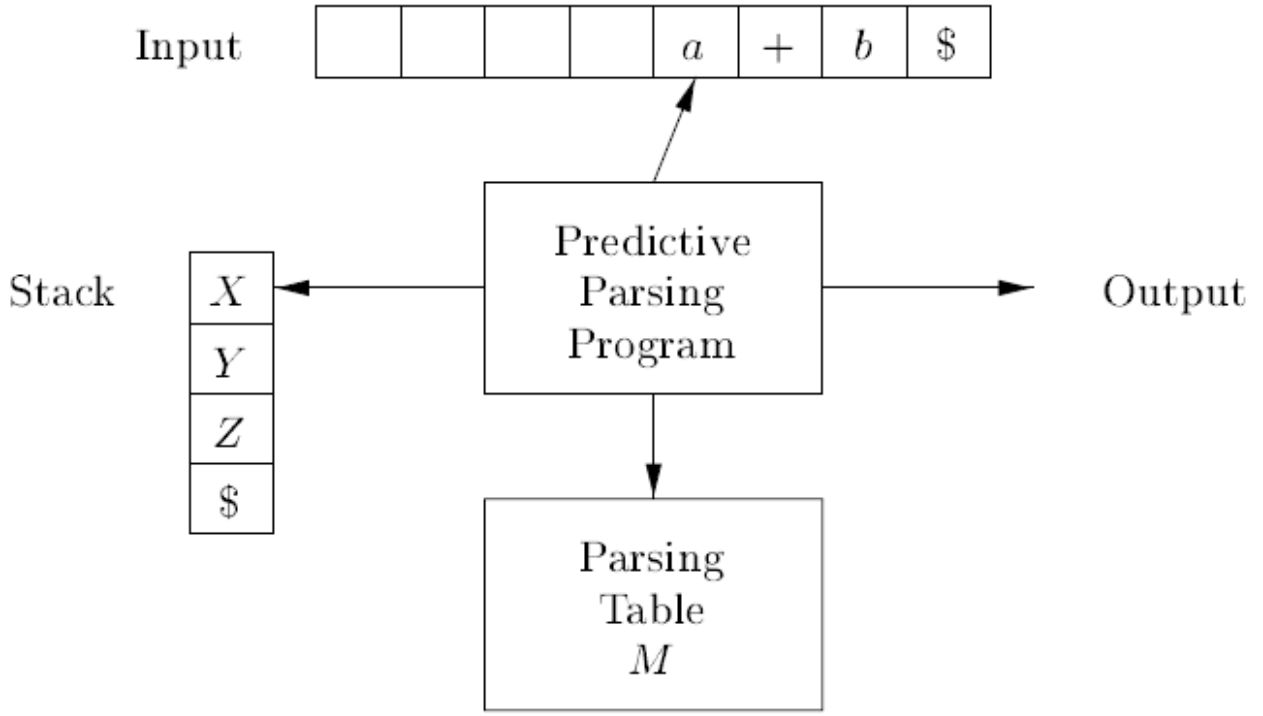


Figure 4.19: Model of a table-driven predictive parser

Parsing Process

- Grammar: $S \rightarrow (S)S \mid \varepsilon$
- Input string: ( )

	(	)	$
S	S→(S)S	S→ε	S→ε

Steps	Parsing Stack	Input	Action
1	$S	( )$	S → (S)S
2	$S)S(	( )$	match
3	$S)S	)$	S → ε
4	$S)	)$	match
5	$S	$	S → ε
6	$	$	accept

Generate

Match

$$S \Rightarrow (S)S \Rightarrow ( )S \Rightarrow ( )$$

- 精准地对应了最左推导。
- 这是自顶向下分析方法的特点。

Configuration

Stack	Input	Action
$S	InputString$	generate
…		
$…X	a…$	$X \in V_N$, generate; $X \in V_T$, match
…		
$	$	accept

Steps	Parsing Stack	Input	Action
1	$E	$i_1*i_2+i_3$	$E \rightarrow TE'$
2	$E'T	$i_1*i_2+i_3$	$T \rightarrow FT'$
3	$E'T'F	$i_1*i_2+i_3$	$F \rightarrow i$
4	$E'T'i	$i_1*i_2+i_3$	match
5	$E'T'	$*i_2+i_3$	$T' \rightarrow *FT'$
6	$E'T'F*	$*i_2+i_3$	match
7	$E'T'F	i_2+i_3	$F \rightarrow i$
8	$E'T'i	i_2+i_3	match
9	$E'T'	$+i_3$	$T' \rightarrow \varepsilon$
10	$E'	$+i_3$	$E' \rightarrow +TE'$

	i	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E′		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T′		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

Example 4.35

- 推导中的各个句型对应于：已经被匹配的输入部分 + 栈的内容。

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \to TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
	$\mathbf{id}\ T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $T' \to \epsilon$
$\mathbf{id}$	$+\ TE'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $E' \to +\ TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
$\mathbf{id} +$	$\mathbf{id}\ T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \to * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id}\ T'E'\$$	$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \to \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \to \epsilon$

Figure 4.21: Moves made by a predictive parser on input $\mathbf{id} + \mathbf{id} * \mathbf{id}$

- Can an $\mathrm{LL}(1)$ grammar be ambiguous ?
- Can an ambiguous grammar be $\mathrm{LL}(1)$?
- Can an unambiguous grammar be $\mathrm{LL}(1)$?
- Can a left-recursive grammar be $\mathrm{LL}(1)$?
- Not all non-LL(1) grammars can be rewritten into LL(1) grammars.

4.5 Bottom-Up Parsing

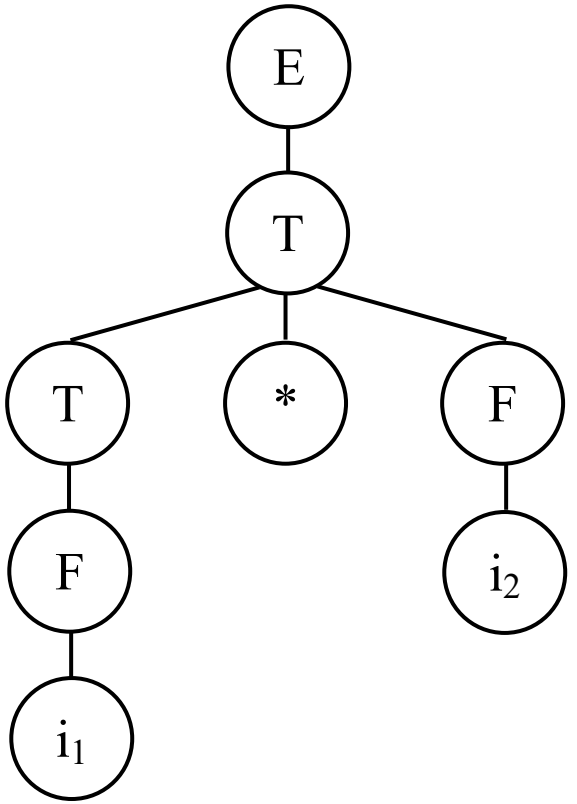
- 从分析树的叶节点开始，逐步向上创建结点，直至创建出根结点。
- 从给定的输入串开始，根据文法规则逐步进行归约，直至归约出开始符号。
- LR分析，算符优先分析

Example

- Left recursion is not a problem in bottom-up parsing.

$$E \rightarrow E{+}T \mid T$$
$$T \rightarrow T{*}F \mid F$$
$$F \rightarrow (E) \mid \mathbf{i}$$

$$\begin{aligned} E &\Rightarrow T \\ &\Rightarrow T{*}F \\ &\Rightarrow T{*}i_2 \\ &\Rightarrow F{*}i_2 \\ &\Rightarrow i_1{*}i_2 \end{aligned}$$



4.5.1 Shift and Reduce

- 把输入符号一个一个地移进栈里。
- 当栈顶形成某个产生式A → α右侧的整个符号串α时，把栈顶的α归约成A 。

G(S)
(1) S → aAcBe
(2) A → b
(3) A → Ab
(4) B → d

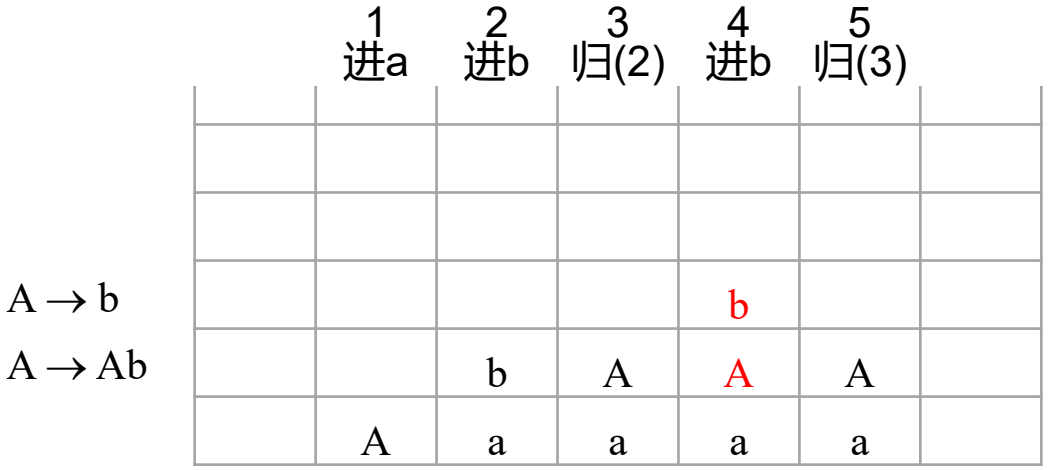
Input: abbcde

	1 进a	2 进b	3 归(2)	4 进b	5 归(3)	6 进c	7 进d	8 归(4)	9 进e	10 归(1)
									e	
							d	B	B	
				b		c	c	c	c	
		b	A	A	A	A	A	A	A	
	a	a	a	a	a	a	a	a	a	

- 对“可归约串”的定义

4.5.2 Handle

- 非正式地讲，句柄是和某个产生式右侧的符号串匹配的子串。



$E \to E+T \mid T$

$T \to T*F \mid F$

$F \to (E) \mid \mathbf{id}$

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \to \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \to F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \to \mathbf{id}$
$T * F$	$T * F$	$T \to T * F$
T	T	$E \to T$

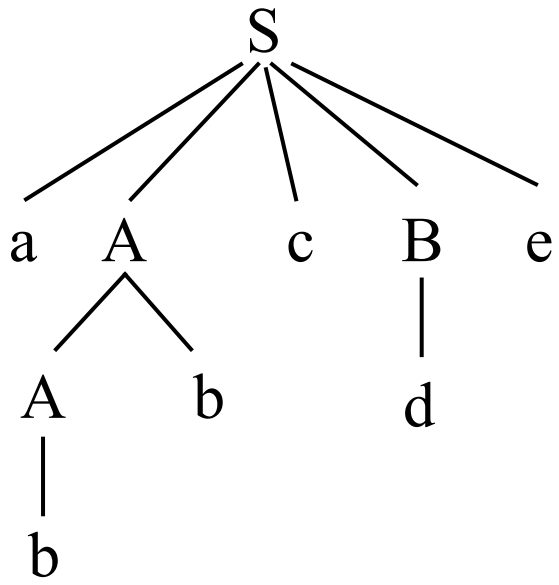
Figure 4.26: Handles during a parse of $\mathbf{id}_1 * \mathbf{id}_2$

- Formally, if $S \Rightarrow^*_{rm} \alpha A \omega \Rightarrow_{rm} \alpha\beta\omega$, then production $A \to \beta$ in the position following α is a **handle** of $\alpha\beta\omega$.
- Alternatively, a handle of a right-sentential form γ (= $\alpha\beta\omega$) contains three elements:
  - a production $A \to \beta$,
  - a position of γ where the string β may be found,
  - after replacing β by A, $\alpha A \omega$ is still a right sentential form.

Handle Pruning

- 通过句柄剪枝可以得到一个反向的最右推导（规范推导）。

$$S \Rightarrow aAcBe \Rightarrow aAcde \Rightarrow aAbcde \Rightarrow abbcde$$



Right-Sentential Form & Handle	Reducing Production
a<u>b</u>bcde	A → b
a<u>Ab</u>cde	A → Ab
aAc<u>de</u>	B → d
<u>aAcBe</u>	S → aAcBe

- G(S)：　S → a | (T)　　T → T,S | S　　string: ((T),a)

4.5.3 Shift-Reduce Parsing

- Consider the grammar:

  $E \rightarrow E+T \mid T$

  $T \rightarrow T*F \mid F$

  $F \rightarrow (E) \mid i$

- Input: $i_1*i_2+i_3$
- 符号栈内容 + 剩余的输入串 = 当前句型
- 整个归约过程是最右推导的反序。
- 句柄总是出现在栈的顶部。

	Stack	Input	Action
1	$	$i_1*i_2+i_3$$	shift
2	$$i_1$	$*i_2+i_3$$	reduce by F→i
3	$F	$*i_2+i_3$$	reduce by T→F
4	$T	$*i_2+i_3$$	shift
5	$T*	$i_2+i_3$$	shift
6	$T*$i_2$	$+i_3$$	reduce by F→i
7	$T*F	$+i_3$$	reduce by T→T*F
8	$T	$+i_3$$	reduce by E→T
9	$E	$+i_3$$	shift
10	$E+	$i_3$$	shift
11	$E+$i_3$	$$	reduce by F→i
12	$E+F	$$	reduce by T→F
13	$E+T	$$	reduce by E→E+T
14	$E	$$	accept

Actions & Configuration

- Actions

shift	读入下一个输入符号并把它移进栈。
reduce	当栈顶符号串形成句柄时进行归约，用产生式左侧的非终结符替换栈顶的句柄。
accept	当栈内只有 $ 和开始符号，输入串中也只有 $ 时，分析成功，是一种特殊的归约。
error	发现一个语法错误，并调用出错处理程序。

- Configuration

Stack	Input	Action
$	InputString$	shift/reduce
…	…	shift/reduce
$S	$	accept

符号栈内容 + 剩余的输入串内容 = 当前句型

4.6 Introduction to LR Parsing: Simple LR

- LR(0), SLR(1)
  - “L” : processes the input from left to right;
  - “R” : rightmost derivation in reverse;
  - “0/1” : zero or one symbol of lookahead is used.
  - “S” : Simple
- Canonical LR methods
  - Rightmost derivations are sometimes called canonical derivations.

4.6.1 Why LR Parser?

- Bottom-up parsing algorithms are in general more powerful than top-down methods. For example, left recursion is not a problem in bottom-up parsing.
- Indeed, all of the important bottom-up methods are really too complex for hand coding.
- A specialized tool, an LR parser generator, is needed. For example, Yacc.

4.6.2 Items and the LR(0) Automaton

- Shift-reduce parser
  - when to shift and when to reduce ?
- Handle
- The whole analysis process is actually a process of continuous state transition.

	Stack	Input	Action
1	$	$i_1*i_2+i_3\$$	shift
2	$\$i_1$	$*i_2+i_3\$$	reduce by F→i
3	$\$F$	$*i_2+i_3\$$	reduce by T→F
4	$\$T$	$*i_2+i_3\$$	shift
5	$\$T*$	$i_2+i_3\$$	shift
6	$\$T*i_2$	$+i_3\$$	reduce by F→i
7	$\$T*F$	$+i_3\$$	reduce by T→T*F
8	$\$T$	$+i_3\$$	reduce by E→T
9	$E	$+i_3\$$	shift
10	$E+	$i_3\$$	shift
11	$\$E+i_3$	$	reduce by F→i
12	$E+F	$	reduce by T→F
13	$E+T	$	reduce by E→E+T
14	$E	$	accept

Items

- 文法的LR(0)项是G的产生式右侧的符号串中带有一个圆点。

$A \to XYZ$: $A \to .\ XYZ$ $A \to X.YZ$ $A \to XY.Z$ $A \to XYZ.$

$A \to \varepsilon$: $A \to .$

归约项目	$A \to \alpha.$ (complete item)
接受项目	$S \to \alpha.$ (特殊的归约项目)
移进项目	$A \to \alpha.a\beta$ $(a \in V_T)$
待约项目	$A \to \alpha\ .B\beta$ $(B \in V_N)$
Initial item	$A \to .\alpha$

- LR(0)项目可以用来表示分析过程中的不同状态。

- $E \to (L) \mid \mathbf{a}$

  $L \to L,E \mid E$

LR(0)分析：

- 对于文法G，构造一个NFA；
- 把NFA确定化，得到一个DFA；
- 把DFA转变为LR分析表；
- 根据LR分析表进行分析。

对于文法G，构造一个NFA

- An NFA consists of $<S, \Sigma, f, s_0, F>$

  S ： LR(0)项　　Σ: V_T & V_N

  s_0 : augment the grammar. The start state is unique.

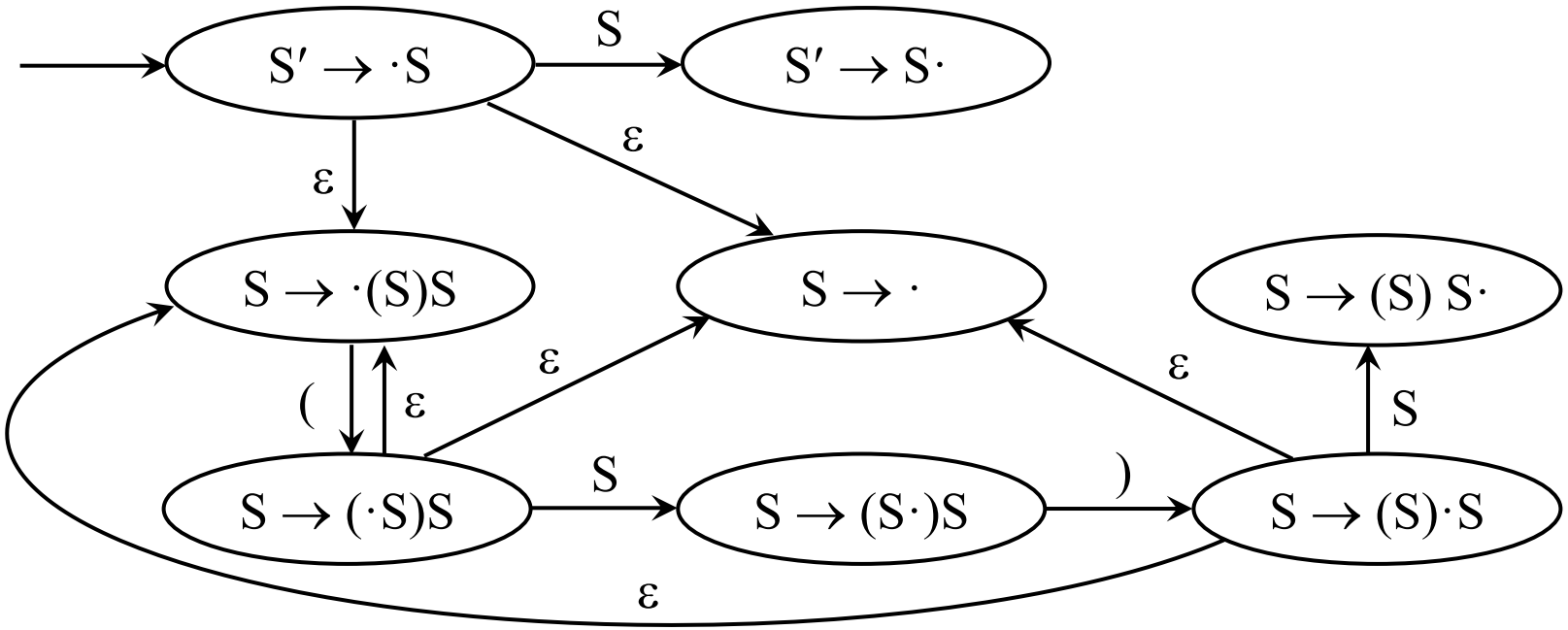
  f: How to add transitions ?　　How to add ε-transitions ?　　Why add ε-transitions like this?

  若状态 ***i*** 为$X \to \alpha.A\beta$，A为非终结符，则从状态 ***i*** 画一条 ε 边到所有状态$A \to .\gamma$。

  F: 该NFA没有接受状态。分析程序决定何时接受：当且仅当使用$S' \to S$进行归约，且输入符号串也为空时，输入符号串被接受。

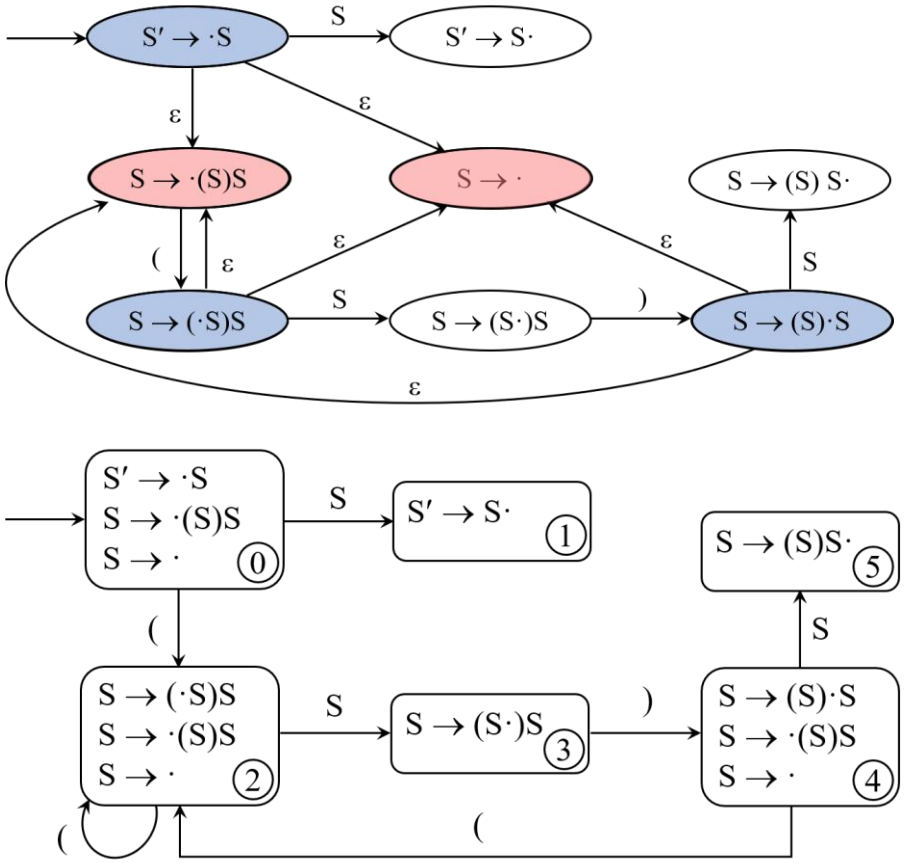
$S' \to S$

$S \to (S)S \mid \varepsilon$



把NFA确定化，得到一个DFA（参考）

- subset construction



State	S	(	)
S′ →·S S →·(S)S S →·	S′ →S.	S → (·S)S S →·(S)S S →·	
S′ →S.			
S → (·S)S S →·(S)S S →·	S→ (S·)S	S → (·S)S S →·(S)S S →·	
S → (S·)S			S → (S)·S S →·(S)S S →·
S → (S)·S S →·(S)S S →·	S → (S)S.	S → (·S)S S →·(S)S S →·	
S → (S)S.			

- 这个过程能不能简化?

CLOSURE and GOTO

- 如果 I 是文法G的一个项集，则根据以下两个规则构造 CLOSURE(I):
  1. 将 I 中的每个项加入 CLOSURE(I)；
  2. 如果$A \to \alpha \cdot B\beta$ 在CLOSURE(I)中，$B \to \gamma$ 是一个产生式， 并且项$B \to \cdot\gamma$ 不在 CLOSURE(I)中，则将该项加入其中。 重复应用该规则，直到没有新的项加入CLOSURE(I)为止。
- GOTO函数用于定义状态转移。

  GOTO (I, X) = CLOSURE(move(I, X))

LR(0)分析

- 对于文法G，构造一个NFA；
- 把NFA确定化，得到一个DFA；
- 把DFA转变为LR分析表；
- 根据LR分析表进行分析。

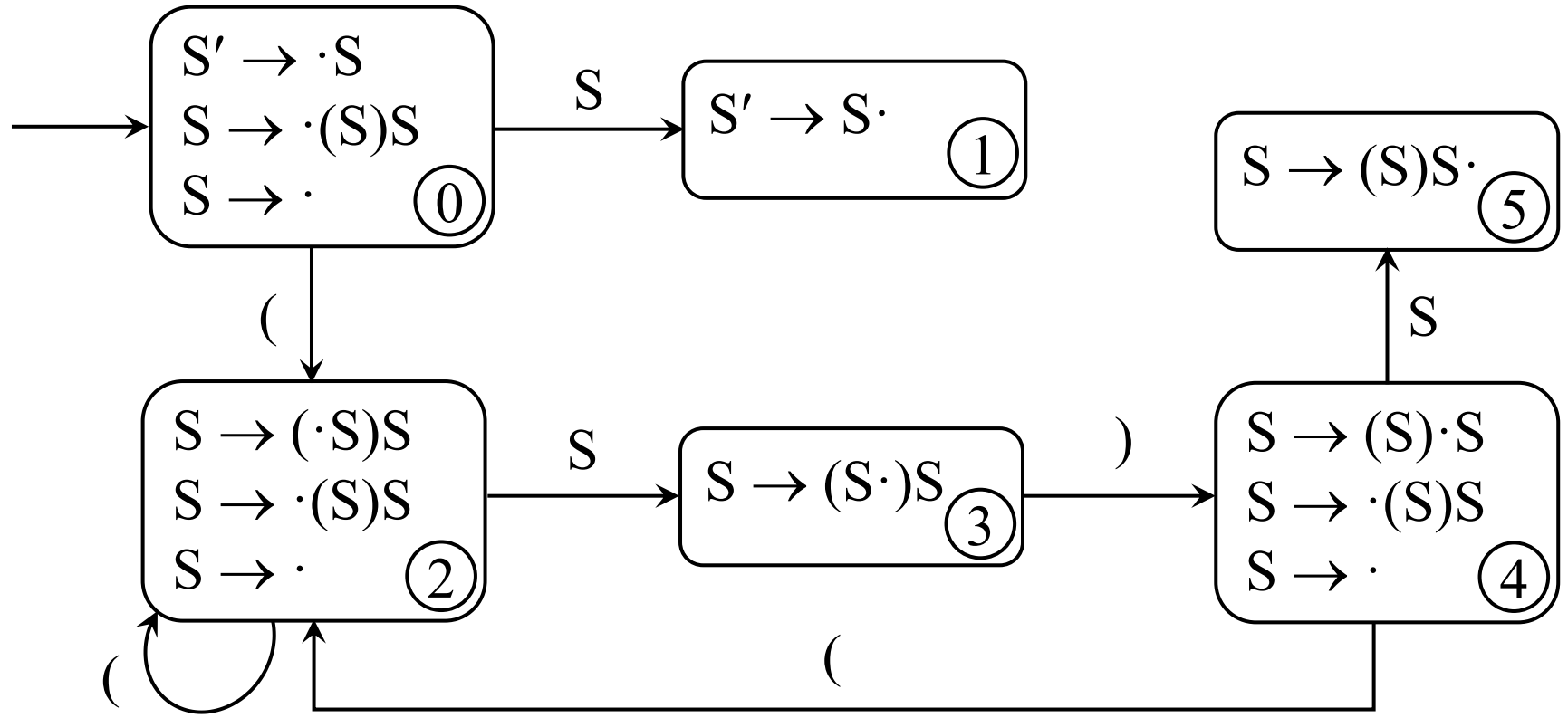
LR(0)分析

- 对于文法G，构造一个DFA，即LR(0)自动机；
- 把DFA转变为LR分析表；
- 根据LR分析表进行分析。

LR(0) Automaton

- LR(0) 项集（DFA state）
- LR(0) 项集族（规范LR(0)项集族）

$S' \to S$　　$S \to (S)S \mid \varepsilon$



Example 4.40 & 4.41

- 构造表达式文法的规范LR(0)项目集族。

$S' \to E$

$E \to E+T \mid T$

$T \to T*F \mid F$

$F \to (E) \mid id$

I_0: $S' \to \cdot E$
$E \to \cdot E+T$
$E \to \cdot T$
$T \to \cdot T*F$
$T \to \cdot F$
$F \to \cdot (E)$
$F \to \cdot i$

I_1: $S' \to E \cdot$
$E \to E \cdot +T$

I_2: $E \to T \cdot$
$T \to T \cdot *F$

I_3: $T \to F \cdot$

I_4: $F \to (\cdot E)$
$E \to \cdot E+T$
$E \to \cdot T$
$T \to \cdot T*F$
$T \to \cdot F$
$F \to \cdot (E)$
$F \to \cdot i$

I_5: $F \to i \cdot$

I_6: $E \to E+\cdot T$
$T \to \cdot T*F$
$T \to \cdot F$
$F \to \cdot (E)$
$F \to \cdot i$

I_7: $T \to T*\cdot F$
$F \to \cdot (E)$
$F \to \cdot i$

I_8: $F \to (E \cdot)$
$E \to E \cdot +T$

I_9: $E \to E+T \cdot$
$T \to T \cdot *F$

I_{10}: $T \to T*F \cdot$

I_{11}: $F \to (E) \cdot$

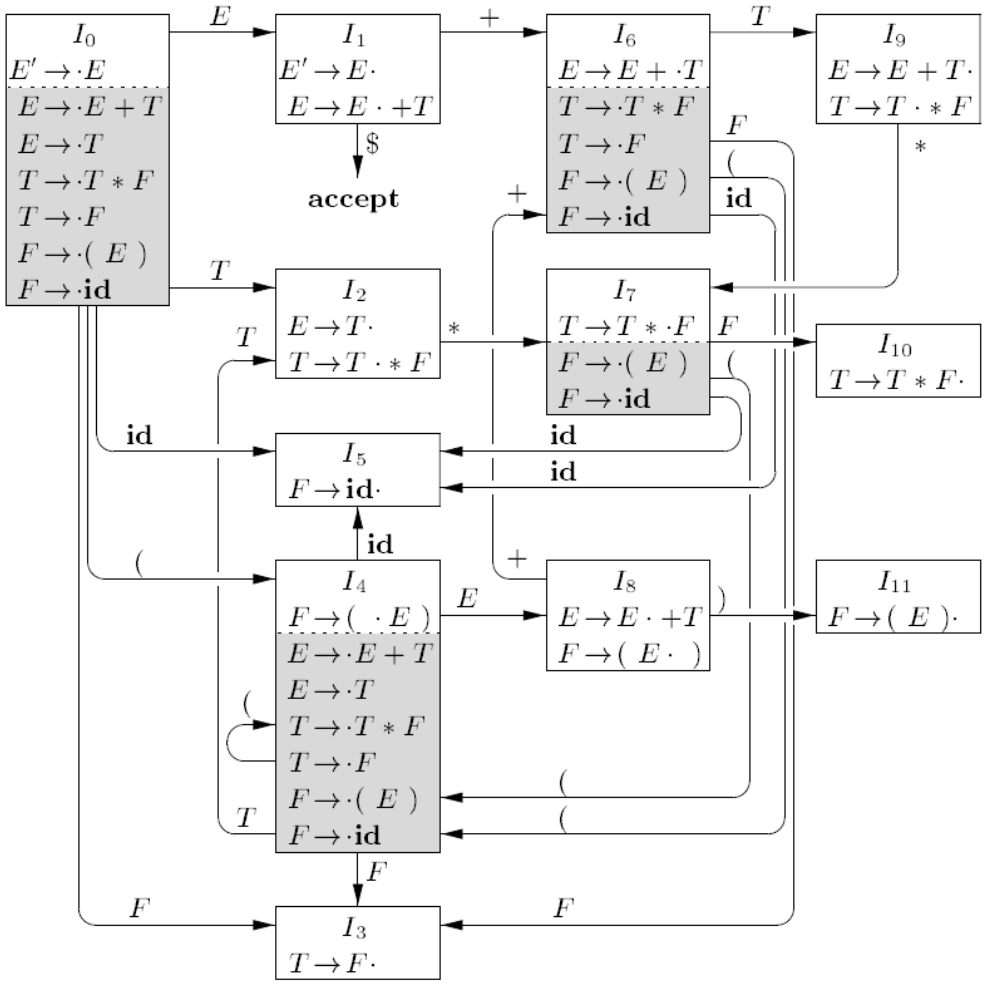


Figure 4.31: LR(0) automaton for the expression grammar (4.1)

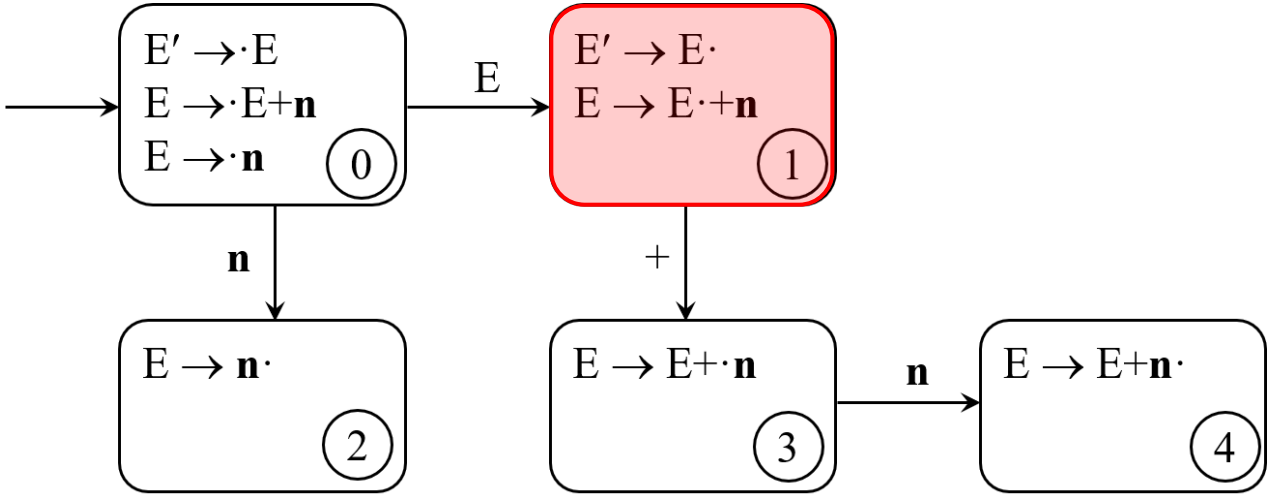
- $A \to (A) \mid \mathbf{a}$

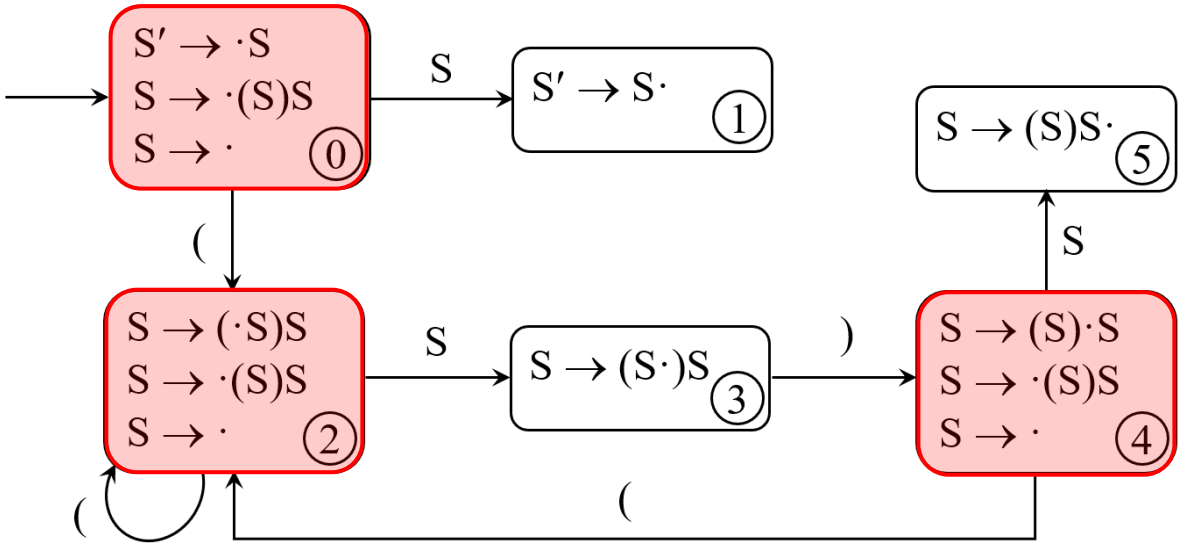
- $E \to E+n \mid \mathbf{n}$

LR(0) Grammar

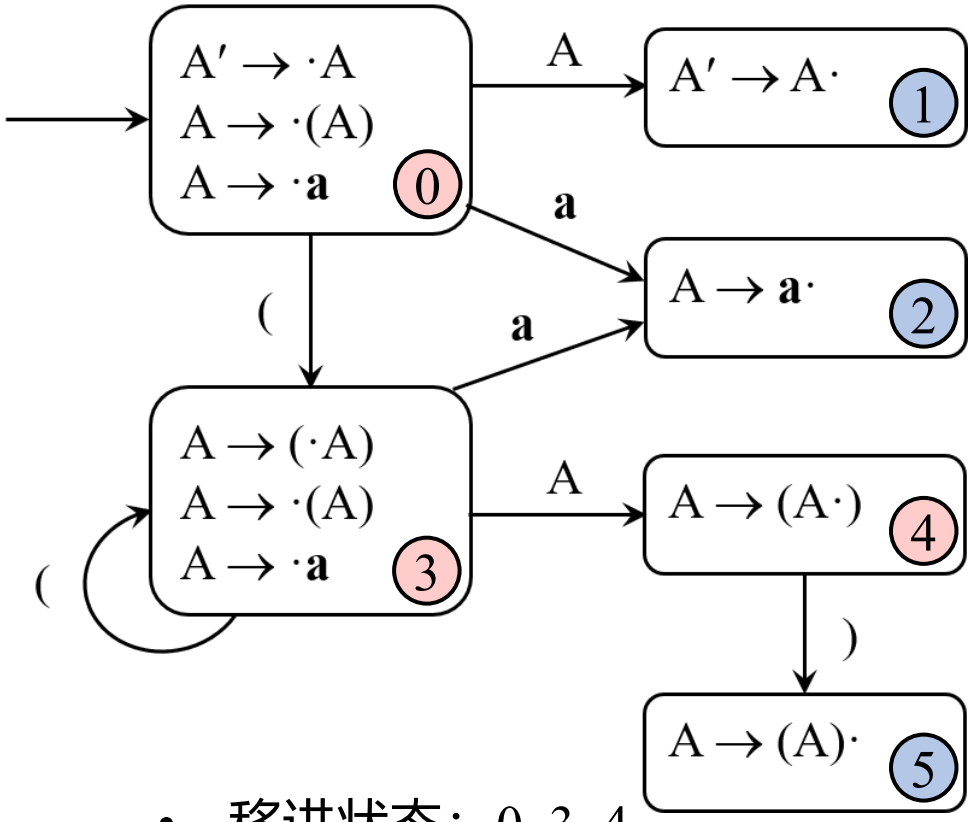
- 如果文法G的LR(0)自动机中的每个状态都不存在下述情况：
  1. 既含移进项目又含归约项目，（移进-归约冲突）
  2. 含有多个归约项目，（归约-归约冲突）

  则称G是一个LR(0)文法。





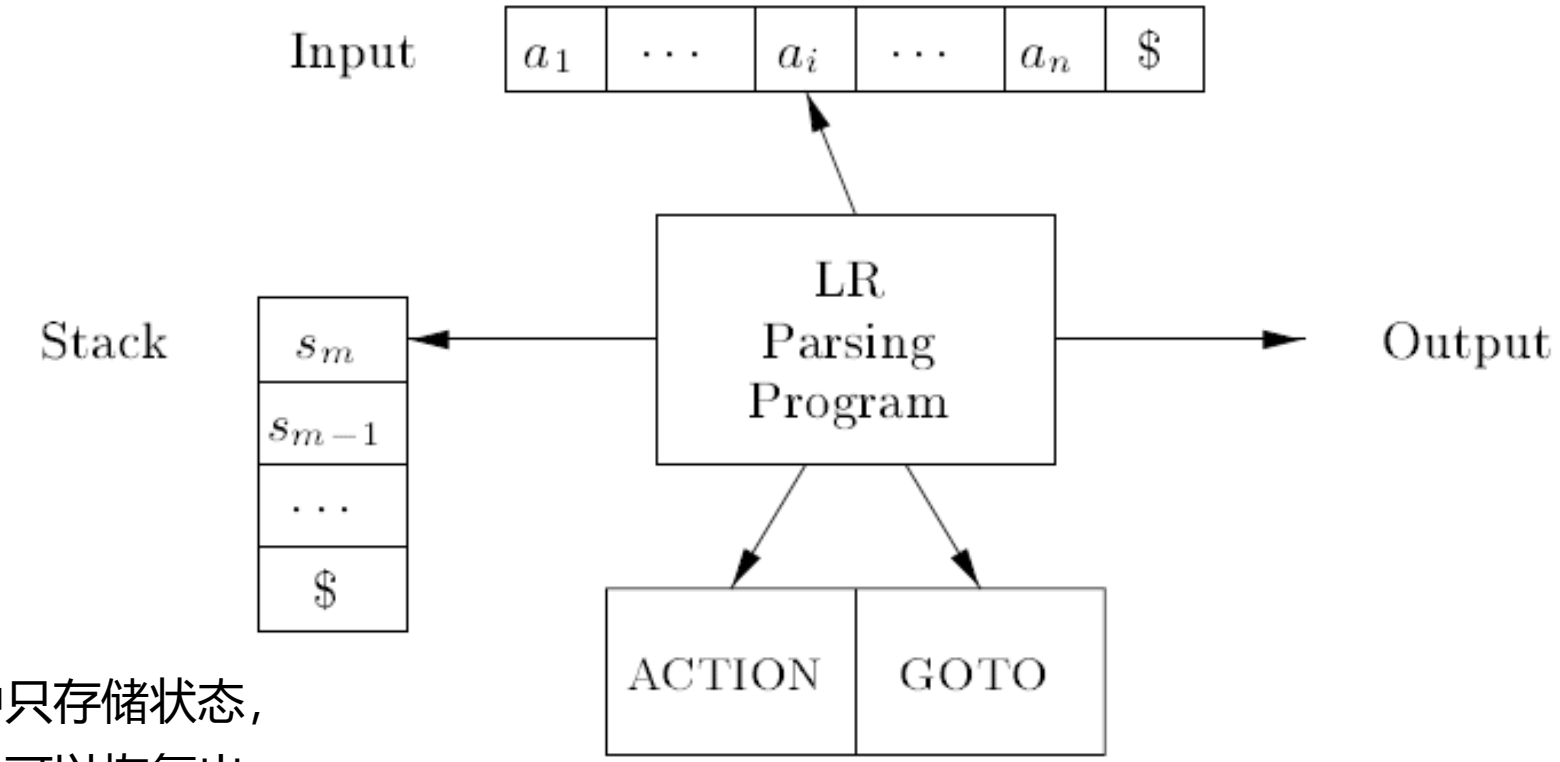
Use of the LR(0) Automaton



- 移进状态：0, 3, 4
- 归约状态：1, 2, 5
- Why LR(0)？

	State	Symbol	Input	Action
1	$ 0	$	((a)) $	shift
2	$ 0 3	$ (	(a)) $	shift
3	$ 0 3 3	$ ((	a)) $	shift
4	$ 0 3 3 2	$ ((a	)) $	reduce A → a
5	$ 0 3 3 4	$ ((A	)) $	shift
6	$ 0 3 3 4 5	$ ((A)	) $	reduce A → (A)
7	$ 0 3 4	$ (A	) $	shift
8	$ 0 3 4 5	$ (A)	$	reduce A → (A)
9	$ 0 1	$ A	$	accept

4.6.3 The LR-Parsing Algorithm

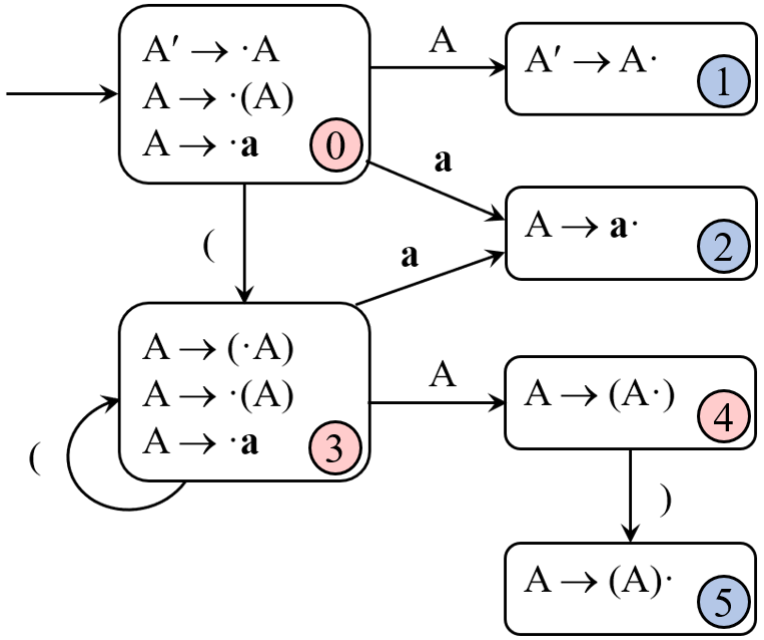


- 分析栈中只存储状态，从状态中可以恢复出相应的文法符号串。

Figure 4.35: Model of an LR parser

Construction of LR Parsing Table

- 把LR(0)自动机的信息写入LR分析表中。设 I_k 为当前状态，
  1. 若$A \to \alpha. \in I_k$，则对所有终结符a(或$)， ACTION[k,a] = r(A→α)。
  2. 若$S' \to S. \in I_k$，则ACTION[k,$] = acc。
  3. 若$A \to \alpha.a\beta \in I_k$，$a \in V_T$，且$GOTO(I_k, a) = I_j$，则ACTION[k,a] = sj。
  4. 若$A \to \alpha.B\beta \in I_k$，$B \in V_N$，且$GOTO(I_k, B) = I_j$，则GOTO[k,B] = j。
  5. 分析表中剩余的空白格置为出错标志。



- 无移进-归约冲突，无归约-归约冲突

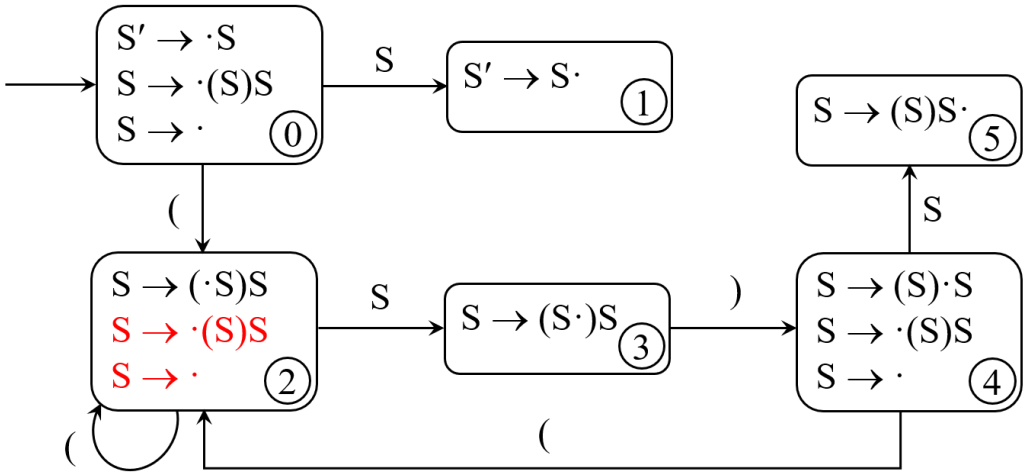
State	ACTION				GOTO
	a	(	)	$	A
0	s2	s3			1
1					
2	r(A→a)	r(A→a)	r(A→a)	r(A→a)	
3	s2	s3			4
4			s5		
5	r(A→(A))	r(A→(A))	r(A→(A))	r(A→(A))	

State	ACTION				GOTO
	a	(	)	$	A
0	s2	s3			1
1				acc	
2	r(A→a)	r(A→a)	r(A→a)	r(A→a)	
3	s2	s3			4
4			s5		
5	r(A→(A))	r(A→(A))	r(A→(A))	r(A→(A))	

	State	Symbol	Input	Action
1	$ 0	$	((a)) $	s3
2	$ 0 3	$ (	(a)) $	s3
3	$ 0 3 3	$ ((	a)) $	s2
4	$ 0 3 3 2	$ ((a	)) $	r(A→a)
5	$ 0 3 3 4	$ ((A	)) $	s5
6	$ 0 3 3 4 5	$ ((A)	) $	r(A→(A))
7	$ 0 3 4	$ (A	) $	s5
8	$ 0 3 4 5	$ (A)	$	r(A→(A))
9	$ 0 1	$ A	$	accept

Problem & Solution

- LR(0)文法没有实用价值。



State	ACTION			GOTO
	(	)	$	S
2	s2, r(S→ε)	r(S→ε)	r(S→ε)	3

- SLR(1)：解决冲突的时候需要lookahead一个token。

4.6.4 Construction of SLR(1) Parsing Table

- 构造SLR(1)分析表

  设I_k为当前状态，

  1. 若$A \to \alpha. \in I_k$，则对所有$a \in FOLLOW(A)$， ACTION[k,a] = r($A \to \alpha$)。
  2. 若$S' \to S. \in I_k$，则ACTION[k,$] = acc。
  3. 若$A \to \alpha.a\beta \in I_k$， $a \in V_T$，且$GOTO(I_k, a) = I_j$，则ACTION[k,a] = sj。
  4. 若$A \to \alpha.B\beta \in I_k$， $B \in V_N$，且$GOTO(I_k, B) = I_j$，则GOTO[k,B] = j。
  5. 分析表中剩余的空白格置为出错标志。

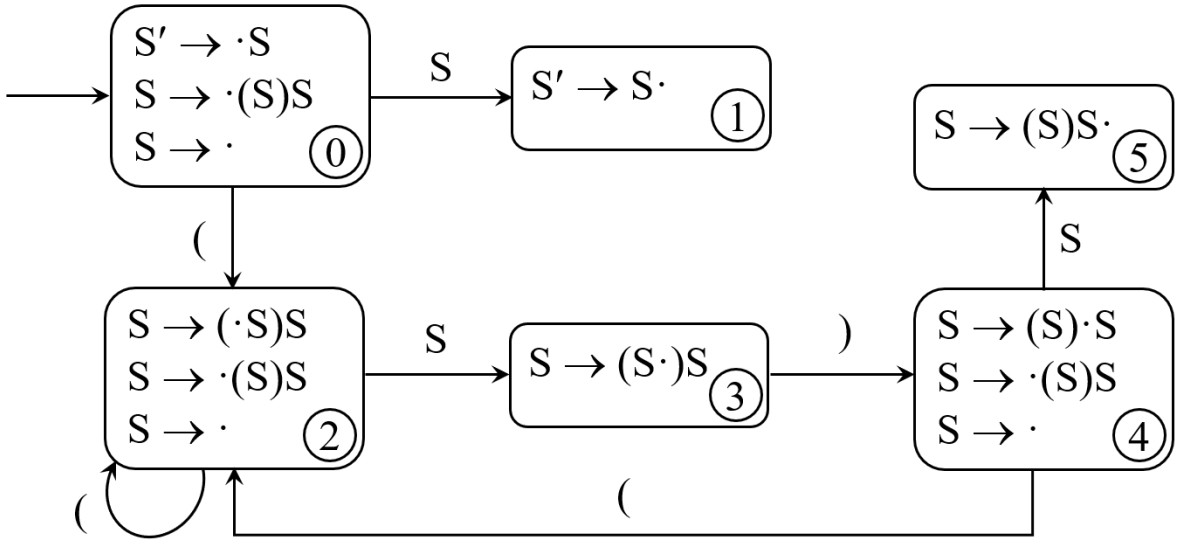
- 构造LR(0)分析表

  设I_k为当前状态，

  1. 若$A \to \alpha. \in I_k$，则对所有终结符a(或$)， ACTION[k,a] = r($A \to \alpha$)。
  2. 若$S' \to S. \in I_k$，则ACTION[k,$] = acc。
  3. 若$A \to \alpha.a\beta \in I_k$， $a \in V_T$，且$GOTO(I_k, a) = I_j$，则ACTION[k,a] = sj。
  4. 若$A \to \alpha.B\beta \in I_k$， $B \in V_N$，且$GOTO(I_k, B) = I_j$，则GOTO[k,B] = j。
  5. 分析表中剩余的空白格置为出错标志。

SLR(1) Grammar

- A grammar is an SLR(1) grammar if there is no conflict in the SLR-parsing table.
- FOLLOW(S′) = { $ } FOLLOW(S) = { $,) }



State	ACTION			GOTO
	(	)	$	S
0				
1				
2				
3				
4				
5				

Example 4.45

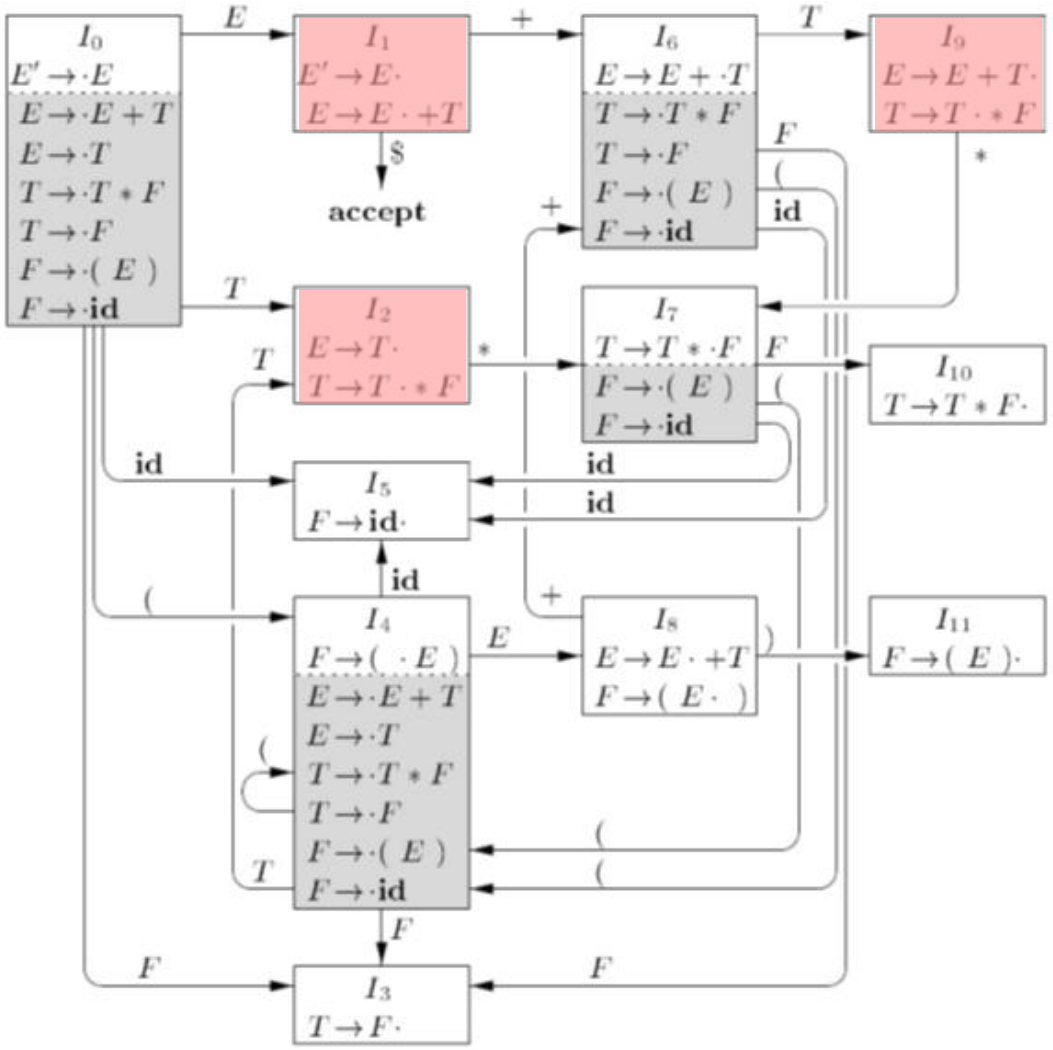


Figure 4.31: LR(0) automaton for the expression grammar (4.1)

- FOLLOW(E′) = { $ } FOLLOW(E) = { $, +,) }
- r 后面的数字表示产生式规则的编号，不是状态编号。

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Figure 4.37: Parsing table for expression grammar

SLR(1)文法 vs. 无二义性文法

- SLR(1)文法是无二义性文法。
- 无二义性文法是不是SLR(1)文法?
- SLR(1)的局限
  - Example 4.48　　S → L=R | R
    L → *R | id
    R → L

I_0: $S' \to \cdot S$
$S \to \cdot L = R$
$S \to \cdot R$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$
$R \to \cdot L$

I_1: $S' \to S\cdot$

I_2: $S \to L\cdot = R$
$R \to L\cdot$

I_3: $S \to R\cdot$

I_4: $L \to *\cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_5: $L \to \mathbf{id}\cdot$

I_6: $S \to L = \cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_7: $L \to *R\cdot$

I_8: $R \to L\cdot$

I_9: $S \to L = R\cdot$

Figure 4.39: Canonical LR(0) collection for grammar (4.49)

- FOLLOW(A)是指所有可能推导出的句型中，可以跟随在A后的终结符号集；
- LR分析法的归约应该仅对右句型中跟随在A后的终结符有效。否则，归约之后得到的符号串可能不是右句型。(计算FOLLOW集合所得到的lookahead符号集可能大于实际能够出现的lookahead符号集。)
- 寻找新的lookahead符号来解决：LR(1)分析、LALR(1)分析

End of Chapter Four